🏫 디미고(한국디지털미디어고등학교) 소질·적성검사 완벽 가이드

> **중학생의 눈높이에 맞춘 쉽고 재미있는 입시 준비!**

📌 진짜 2단계 지필 시험은 무엇일까요?

디미고 일반전형에서는 1단계에서 여러분의 중학교 내신(국어, 수학, 영어)을 봅니다.
하지만 1단계를 통과하고 **학교에 직접 가서 치르는 진짜 시험(2단계 지필고사)**은 국영수가 아닙니다!

바로 **소질·적성검사** 인데요, 여러분이 IT 인재로서 얼마나 잘 성장할 수 있는지 두뇌의 잠재력을 보는 시험입니다.

⏰ 시험 형식

- **문항 수**: 총 30문항 (객관식)
- **시험 시간**: 50분
- **핵심**: 50분 안에 30문제를 풀어야 하므로, 1문제당 **약 1분 30초** 안에 푸는 연습이 필수입니다!

🎯 무엇을 물어보나요? (출제 3대장)

영역	무엇을 평가하나요?	어떤 문제가 나오나요? (예시)
1. 논리적 추론	규칙을 찾고 논리적으로 생각하는 힘	"철수는 거짓말을 하고 영희는 참말을 한다면, 범인은 누구일까?" 같은 명제/퍼즐 문제
2. 이산수학	컴퓨터가 좋아하는 수학 (정수, 경우의 수)	경우의 수, 이진수(0과 1) 변환, 점과 선으로 이루어진 그래프 문제

영역	무엇을 평가하나요?	어떤 문제가 나오나요? (예시)
3. 컴퓨팅 사고력	문제를 컴퓨터처럼 단계별로 해결하는 능력	순서도(Flowchart) 읽기, 알고리즘의 규칙 찾기, 데이터 정리하는 방법

📚 교재 구성 (4단계 합격 로드맵)

중학생 여러분이 학원 없이도 스스로 완벽하게 준비할 수 있도록, 체계적인 4단계 커리큘럼으로 구성했습니다. 아래 순서대로 차근차근 학습해 보세요!

단계	구분	주요 내용 및 학습 목표
1단계	**입시 가이드 및 용어 사전**	• 입시 개요 및 전형별 합격 전략 안내 • IT 및 소질·적성 통합 용어사전 (컴퓨터 인프라, 프로그래밍, 논리·이산수학, 보안·AI)
2단계	**영역별 기본 개념 정복**	• 논리적 추론 (명제, 참/거짓, 조건 추리) • 이산수학 (진법, 경우의 수, 그래프 이론) • 컴퓨팅 사고력 (순서도, 시간복잡도, 탐색 기법)
3단계	**100점 만점 심화 과정**	• 만점 달성 전략 및 고난도 대비법 • 비버챌린지 집중 공략 • 정렬 알고리즘 특강 및 필수 회로 특강
4단계	**35분 실전 모의고사 (6회)**	• 35분 실전 타이머 훈련 (총 6세트 126문항) • 실전 문제지와 단계별 상세 정답·해설지 분권 구성

💡 선배들이 알려주는 합격 꿀팁!

1. **포기할 문제는 빨리 포기하라!**
    - 30문제를 50분에 풀어야 해요. 2분 이상 고민해도 답이 안 보이는 복잡한 퍼즐 문제는 과감히 찍고(또는 넘기고) 쉬운 문제부터 푸세요.
2. **손으로 직접 그려보세요.**
    - 머리로만 생각하면 꼬입니다. 시험지에 표를 그리거나 동그라미를 치면서 조건들을 정리하면 훨씬 쉽게 풀려요.

3. **정보올림피아드(비버 챌린지) 문제를 참고하세요.**
    - 디미고 시험은 '비버 챌린지(Bebras Challenge)'나 정보올림피아드 1차 예선의 이산수학 문제들과 유형이 비슷합니다.

자, 그럼 첫 번째 영역인 **[논리적 추론]** 부터 시작해 볼까요? 화이팅! 🚀

📖 디미고 입시 완벽 대비: IT 및 소질·적성 통합 용어사전

> **"교재를 펼치기 전, 이 단어들의 뜻만 알아도 필기시험과 면접의 절반은 쉬워집니다!"**
> 컴퓨터 하드웨어·네트워크·프로그래밍 기초부터 소질·적성검사(논리·이산수학·알고리즘) 및 최신 AI·보안·비즈니스 트렌드까지, 디미고 입시를 준비하는 중학생과 학부모를 위해 중복 없이 체계적으로 집대성한 공식 통합 용어사전입니다.

📚 전체 목차 (Table of Contents)

- **제1편: 컴퓨터 시스템 및 인프라 기초 (Hardware, Network & Software)**
  - 1.1 컴퓨터 구조 및 하드웨어 (CPU, RAM, SSD/HDD, ROM)
  - 1.2 데이터의 단위 (Bit, Byte, KB, MB, GB, TB)
  - 1.3 네트워크와 인터넷 (서버/클라이언트, IP 주소와 포트, 도메인 네임/URL, HTTP/HTTPS, 클라우드 컴퓨팅)
  - 1.4 소프트웨어 및 운영체제 (운영체제 OS, 오픈 소스, 애플리케이션, 웹 애플리케이션)
- **제2편: 프로그래밍 및 컴퓨팅 사고력 기초 (Programming & Logic Foundations)**
  - 2.1 컴퓨팅 사고력과 변수·상수 (컴퓨팅 사고력, 변수, 상수)
  - 2.2 함수와 데이터 흐름 (함수, 매개변수와 인자, 반환값)
  - 2.3 제어 구조 (조건문 IF-ELSE, 반복문 FOR/WHILE)
  - 2.4 오류와 디버깅 (버그 Bug vs 오류 Error, 디버깅 Debugging)
  - 2.5 개발 생태계와 웹 (소스 코드와 기계어, 컴파일러 vs 인터프리터, 모듈과 라이브러리, 웹 3총사, 프론트엔드/백엔드)
- **제3편: 소질·적성검사 핵심 이론 (Reasoning, Discrete Math & Algorithms)**
  - 3.1 논리적 추론 용어 (명제, 진리값, 논리 연산자 AND/OR/NOT, 역·이·대우, 삼단논법, 모순)
  - 3.2 이산수학 용어 (이산수학, 진법 변환, 순열과 조합, 비둘기집 원리, 그래프와 트리, 한붓그리기)
  - 3.3 자료구조 핵심 (자료구조의 개념, 스택 LIFO, 큐 FIFO, 재귀 함수)
  - 3.4 알고리즘과 시간 복잡도 (알고리즘의 개념과 중요성, 순서도 Flowchart, 시간 복잡도 Big-O, 복잡도 비교 곡선)

    - 3.5 정렬 및 6대 핵심 탐색 기법 (정렬 Sorting, 선형 탐색, 이진 탐색, 완전 탐색, DFS, BFS, 백트래킹)
- **제4편: 정보 보안 및 최신 AI 테크 트렌드 (Security & Emerging Tech)**
    - 4.1 정보 보안 및 암호 기술 (해킹과 화이트 해커, 악성코드와 랜섬웨어, 방화벽, 대칭키 vs 공개키 암호화, 일방향 해시 암호화)
    - 4.2 인공지능(AI) 핵심 용어 (인공지능 AI, 머신러닝, 딥러닝, 생성형 AI, LLM, AI 에이전트, AGI, 프롬프트 엔지니어링)
    - 4.3 디지털 신기술 (빅데이터 Big Data, 사물인터넷 IoT)
- **제5편: e-비즈니스 및 경영·기술 융합 (e-Business & Management)**
    - 5.1 전자상거래 (E-Commerce - B2B, B2C, C2C)
    - 5.2 핀테크 (FinTech)
    - 5.3 UI / UX (사용자 인터페이스 및 사용자 경험)
    - 5.4 플랫폼 비즈니스 (Platform Business와 네트워크 효과)

🖥️ 제1편: 컴퓨터 시스템 및 인프라 기초

1.1 컴퓨터 구조 및 하드웨어 (Hardware)

- **CPU (중앙처리장치, Central Processing Unit)**
    - **뜻**: 컴퓨터의 **두뇌**입니다. 모든 계산과 판단, 명령어 처리를 총괄하는 가장 핵심적인 부품입니다.
    - *비유*: 사람으로 치면 생각하는 '뇌', 식당으로 치면 요리를 직접 진두지휘하는 '주방장'에 해당합니다.
- **RAM (램, 주기억장치, Random Access Memory)**
    - **뜻**: 컴퓨터가 현재 실행 중인 프로그램과 작업 데이터를 잠시 올려두고 빠르게 읽고 쓰는 **임시 작업 공간**입니다. 전원을 끄면 내용이 모두 사라지는 **휘발성(Volatile)** 메모리입니다.
    - *비유*: 주방장이 요리할 때 재료를 올려놓는 '도마'. 도마가 넓을수록(RAM 용량이 클수록) 여러 요리(작업)를 동시에 쾌적하게 진행할 수 있습니다.
- **SSD / HDD (보조기억장치, Storage)**
    - **뜻**: 운영체제, 설치 프로그램, 문서, 사진, 동영상 등을 영구적으로 안전하게 보관하는 대용량 창고입니다. 전원을 꺼도 내용이 유지되는 **비휘발성(Non-Volatile)** 장치입니다.

    - *비교*: 옛날 방식인 HDD(하드디스크)는 모터로 디스크를 돌려 소음이 있고 느렸지만, 최신 SSD(솔리드 스테이트 드라이브)는 순수 반도체 칩을 사용하여 속도가 수십 배 빠르고 충격에 강합니다.
    - *비유*: 식당의 거대한 '식자재 냉장고'.
- **ROM (롬, Read-Only Memory)**
    - **뜻**: 한 번 기록된 데이터를 오직 '읽기'만 할 수 있고 수정할 수 없는 고정 메모리입니다. 전원이 꺼져도 지워지지 않으며, 컴퓨터를 켤 때 하드웨어 부품들을 깨우고 점검하는 부팅 기본 설정(BIOS/UEFI) 등이 영구적으로 내장되어 있습니다.

1.2 데이터의 단위 (Data Units)

- **비트 (Bit, Binary Digit)**
    - **뜻**: 컴퓨터가 데이터를 표현하고 저장하는 **가장 작은 기본 단위**입니다. 2진수 한 자리(0 또는 1)로 표현되며, 전기가 흐르지 않는 상태(0)와 흐르는 상태(1)를 나타냅니다.
- **바이트 (Byte)**
    - **뜻**: 8개의 비트(Bit)가 하나로 모인 단위입니다 (`1 Byte = 8 Bits`). 보통 영문 알파벳 한 글자, 숫자 한 자리를 저장하는 데 1바이트가 필요합니다.
- **데이터 용량의 계층 단위 (Byte ➔ KB ➔ MB ➔ GB ➔ TB)**
    - **순서**: `Byte < KB(킬로바이트) < MB(메가바이트) < GB(기가바이트) < TB(테라바이트) < PB(페타바이트)`
    - **계산 원리**: 컴퓨터는 2진법 체계를 사용하므로, 각 단위는 정확히 **1,024배(2^{10})** 씩 커집니다.
        - 1 KB = 1,024 Bytes
        - 1 MB = 1,024 KB (고음질 MP3 음악 파일 1곡에 약 5~10 MB)
        - 1 GB = 1,024 MB (고화질 영화 1편에 약 2~4 GB)
        - 1 TB = 1,024 GB (현대 컴퓨터의 기본 보조기억장치 용량)

1.3 네트워크와 인터넷 (Network & Internet)

- **서버 (Server) 와 클라이언트 (Client)**

    - **서버(Server)**: 네트워크 상에서 서비스를 **제공(Serve)**하는 중앙 컴퓨터입니다 (예: 유튜브 동영상을 보관하고 전송해 주는 구글의 대형 컴퓨터).
    - **클라이언트(Client)**: 서비스를 **요청하고 이용(Customer)**하는 사용자의 단말기입니다 (예: 스마트폰으로 유튜브 앱을 켜서 영상을 시청하는 내 스마트폰).
- **IP 주소 (IP Address) 와 포트 (Port)**
    - **IP 주소**: 인터넷 망에 연결된 전 세계 모든 컴퓨터를 고유하게 식별하기 위한 **디지털 네트워크 주소(아파트 동 번호)**입니다 (예: `192.168.0.1`).
    - **포트(Port)**: IP 주소로 해당 컴퓨터에 도착한 뒤, 컴퓨터 안에서 실행 중인 여러 프로그램(웹, 메일, 게임 등) 중 **어떤 프로그램 창구로 들어갈지 안내하는 방 번호**입니다 (예: 일반 웹 서버는 80번, 보안 웹 서버는 443번 포트를 기본으로 사용합니다).
- **도메인 네임 (Domain Name) 과 URL**
    - **도메인 네임**: 복잡한 숫자 조합인 IP 주소를 사람이 외우고 사용하기 쉽도록 문자로 바꾼 **인터넷 문패**입니다 (예: `dimigo.hs.kr`, `google.com`).
    - **URL (Uniform Resource Locator)**: 도메인 네임과 접속 규약(프로토콜), 세부 파일 경로를 모두 포함한 **인터넷 자원의 정확한 전체 주소 표기**입니다 (예: `https://www.dimigo.hs.kr/index.php`).
- **HTTP 와 HTTPS (HyperText Transfer Protocol)**
    - **HTTP**: 전 세계 웹 브라우저와 웹 서버가 웹페이지(HTML 문서, 이미지 등)를 서로 주고받기 위해 사용하는 표준 통신 규칙입니다.
    - **HTTPS**: 기존 HTTP에 보안(Secure) 암호화 통신을 결합한 프로토콜입니다. 사용자가 입력한 아이디, 비밀번호, 카드 번호 등을 암호화하여 전송 도중 해커가 중간에서 엿듣지 못하도록 보호합니다 (웹 브라우저 주소창의 자물쇠 아이콘).
- **클라우드 컴퓨팅 (Cloud Computing)**
    - **뜻**: 개인이나 기업이 비싼 서버 컴퓨터나 저장 장치를 직접 방에 사두지 않고, 인터넷(구름 너머)에 존재하는 대규모 IT 인프라 기업(AWS, 구글 클라우드, 마이크로소프트 등)의 컴퓨터 연산 자원과 저장소를 **원하는 만큼 빌려 쓰고 쓴 만큼 비용을 지불**하는 서비스 모델입니다.

1.4 소프트웨어 및 운영체제 (Software & OS)

- **운영체제 (OS, Operating System)**
    - **뜻**: CPU, 메모리, 저장장치 등 복잡한 하드웨어를 직접 제어하고 관리하며, 사용자가 애플리케이션을 쉽고 편리하게 실행할 수 있도록 연결해 주는 **가장 기본**

적이고 핵심적인 시스템 소프트웨어입니다.
    - *대표 예시*: PC용 운영체제(Windows, macOS, Linux), 스마트폰용 모바일 운영체제(Android, iOS).
- **오픈 소스 (Open Source Software, OSS)**
    - **뜻**: 소프트웨어의 설계도에 해당하는 **소스 코드(Source Code)를 모든 사람에게 공개**하여, 누구나 자유롭게 프로그램을 들여다보고, 기능을 개선하며, 무료로 재배포할 수 있도록 허용한 소프트웨어 생태계입니다 (예: Linux 운영체제, 파이썬 언어, Chromium 등).
- **애플리케이션 (Application / App)**
    - **뜻**: 운영체제 위에서 사용자의 특정한 작업 목적(문서 작성, 메신저 대화, 게임 등)을 돕기 위해 개발된 응용 프로그램입니다 (예: 한글, MS 워드, 카카오톡, 포토샵).
- **웹 애플리케이션 (Web Application)**
    - **뜻**: 컴퓨터나 스마트폰에 별도의 프로그램을 번거롭게 설치할 필요 없이, 웹 브라우저(크롬, 사파리, 엣지 등) 창 안에서 곧바로 실행되는 현대적인 응용 프로그램입니다 (예: 구글 스프레드시트, 노션, 웹메일 등).

⌨️ 제2편: 프로그래밍 및 컴퓨팅 사고력 기초

2.1 컴퓨팅 사고력과 변수·상수 (Foundations)

- **컴퓨팅 사고력 (Computational Thinking)**
    - **뜻**: 컴퓨터 과학의 기본 개념과 원리를 활용하여, 복잡하고 방대한 문제를 효율적으로 분해하고 분석하여 **컴퓨터가 실행 가능한 명확한 해결책으로 설계해 내는 종합적인 사고 능력**입니다.
    - *핵심 4단계*:
        1. **문제 분해(Decomposition)**: 큰 문제를 해결 가능한 작은 조각들로 쪼개기.
        2. **패턴 인식(Pattern Recognition)**: 이전의 경험이나 데이터 속에서 규칙과 공통점 찾기.
        3. **추상화(Abstraction)**: 문제 해결에 불필요한 군더더기를 버리고 핵심 정보만 추출하기.
        4. **알고리즘(Algorithm)**: 순서대로 따라 하면 정답이 나오는 단계별 절차 완성하기.

- **변수 (Variable)**
  - **뜻**: 프로그램이 실행되는 동안 어떤 데이터(숫자, 문자열 등)를 임시로 담아두는 **이름표가 붙은 메모리 상자**입니다. 상자 안의 값은 필요에 따라 언제든지 새로운 값으로 변경될 수 있습니다 (예: `score = 100` ➔ `score = 150`).
- **상수 (Constant)**
  - **뜻**: 변수와 반대로, 한 번 초기값을 지정하면 프로그램이 종료될 때까지 **그 값이 절대로 변하지 않는 고정된 값**입니다 (예: 원주율 `PI = 3.141592`, 하루의 시간 `HOURS_IN_DAY = 24`).

2.2 함수와 데이터 흐름 (Function & Data Flow)

- **함수 (Function) 와 메서드 (Method)**
  - **뜻**: 반복적으로 사용되는 특정한 작업이나 계산 절차를 하나로 묶어둔 **재사용 가능한 독립된 코드 모듈(요리 기계)**입니다.
  - **왜 사용할까요?**: 똑같은 계산 코드를 매번 수십 줄씩 반복 작성할 필요 없이, 함수 이름 하나만 딱 부르면(호출하면) 즉시 실행되므로 코드의 가독성이 높아지고 유지보수가 매우 편리해집니다. (객체에 속해 있는 함수를 '메서드'라고 부릅니다.)
- **매개변수 (Parameter) 와 전달인자 (Argument)**
  - **뜻**: 함수라는 기계가 내부 작업을 수행하기 위해 **외부에서 전달받아 집어넣어 주는 입력 데이터(재료)**입니다.
  - *비유*: 믹서기라는 함수가 동작하기 위해 투입구에 넣는 '사과'와 '우유'.
- **반환값 (Return Value)**
  - **뜻**: 함수 내부의 계산이나 동작이 성공적으로 완료된 후, 해당 함수를 호출했던 원래 자리에 쏙 넘겨주는 **최종 결과 데이터(출력물)**입니다.
  - *비유*: 믹서기에서 사과와 우유를 갈아 완성된 '사과 주스 1잔'.

2.3 제어 구조 (Control Structures)

- **조건문 (Conditional Statement: `IF - ELSE`)**
  - **뜻**: 주어진 조건식의 참(True) 또는 거짓(False) 여부에 따라 실행할 명령어의 흐름을 갈림길처럼 분기하는 문장입니다.

    - *예시*: "만약(`IF`) 시험 점수가 90점 이상이면 '합격', 그렇지 않으면(`ELSE`) '불합격'을 출력하라."
- **반복문 (Loop Statement: `FOR`, `WHILE`)**
    - **뜻**: 정해진 횟수만큼 또는 특정 조건이 참인 동안 명령 블록을 되풀이하여 수행하는 문장입니다.
    - **FOR 루프**: 반복 횟수가 명확히 정해져 있을 때 주로 사용합니다 (예: "1부터 100까지 숫자를 하나씩 더하라").
    - **WHILE 루프**: 특정 조건이 만족되는 동안 계속 반복할 때 주로 사용합니다 (예: "배터리 잔량이 20% 이상인 동안 게임을 계속 실행하라").

2.4 오류와 디버깅 (Error & Debugging)

- **버그 (Bug)**
    - **뜻**: 프로그래머가 작성한 소스 코드나 알고리즘 설계 자체에 숨어 있는 **오타, 논리적 결함, 설계 상의 결점**을 의미합니다. (1940년대 실제 나방 한 마리가 컴퓨터 진공관 회로 사이에 끼어 오작동을 일으켰던 일화에서 유래되었습니다.)
- **오류 (Error)**
    - **뜻**: 코드 속의 버그나 예기치 못한 비정상적 조건(0으로 나누기, 메모리 부족, 잘못된 입력 등)으로 인해 **프로그램이 실행 도중 멈추거나, 비정상 종료되거나, 원치 않는 잘못된 계산 결과를 도출하는 실패 현상**을 말합니다.
    - 💡 **버그와 오류의 차이점 한 줄 요약**: 개발자가 코드 속에 남겨둔 결함의 **원인**이 '버그'라면, 그로 인해 컴퓨터가 실행 중 멈추거나 경고를 띄우는 결과적 **현상**이 바로 '오류'입니다.
- **디버깅 (Debugging)**
    - **뜻**: 프로그램에서 버그를 찾아내어 그 원인을 분석하고, 올바르게 코드를 수정하여 정상 작동하도록 바로잡는 일련의 소프트웨어 수리 과정입니다.

2.5 개발 생태계와 웹 (Ecosystem & Web)

- **소스 코드 (Source Code) 와 기계어 (Machine Code)**
    - **소스 코드**: 프로그래머가 사람이 이해할 수 있는 문자 문법(C, Python, Java 등)으로 작성한 프로그램의 **설계도**입니다.

    - **기계어**: 컴퓨터(CPU)가 직접 전기 신호로 알아듣는 0과 1의 2진수 비트 열입니다.
- **컴파일러 (Compiler) 와 인터프리터 (Interpreter)**
    - **컴파일러**: 작성된 소스 코드 전체를 실행 전에 한 번에 통째로 기계어로 번역하여 실행 파일(`.exe`)을 만드는 번역기입니다 (예: C, C++, Rust). 실행 속도가 매우 빠릅니다.
    - **인터프리터**: 소스 코드를 위에서부터 한 줄씩 실시간으로 통역하듯 해석하며 즉시 실행하는 방식입니다 (예: Python, JavaScript). 개발과 테스트가 편리합니다.
- **모듈 (Module) 과 라이브러리 (Library)**
    - **뜻**: 다른 뛰어난 개발자들이 미리 완벽하게 만들어 검증해 둔 유용한 함수, 클래스, 도구들의 **부품 상자(도서관)**입니다. 처음부터 모든 코드를 새로 짤 필요 없이, 필요한 부품을 가져와 조립(import)하여 개발 시간을 비약적으로 단축합니다.
- **웹 프론트엔드 (Front-end) 와 백엔드 (Back-end)**
    - **프론트엔드**: 사용자가 웹 브라우저에서 눈으로 직접 보고 클릭하며 조작하는 화면 영역(디자인, 버튼, 애니메이션 등)을 개발하는 분야입니다.
    - **백엔드**: 화면 뒤의 서버에서 회원가입 인증, 데이터베이스 조회, 비즈니스 계산 등을 안전하게 처리하는 서버 영역을 개발하는 분야입니다.
- **웹 프로그래밍의 3총사 (HTML, CSS, JavaScript)**
    - **HTML (뼈대)**: 웹페이지의 글과 구조, 구역을 정의하는 언어 (집의 기둥과 벽).
    - **CSS (디자인)**: 웹페이지의 색상, 폰트, 레이아웃을 예쁘게 꾸미는 언어 (집의 벽지와 페인트).
    - **JavaScript (동작)**: 버튼 클릭에 반응하고 데이터를 주고받아 화면을 동적으로 움직이게 하는 언어 (집의 자동문과 엘리베이터).

🧩 제3편: 소질·적성검사 핵심 이론

3.1 논리적 추론 용어 (Logical Reasoning)

- **명제 (Proposition)**
    - **뜻**: 내용이 참(True)인지 거짓(False)인지를 객관적으로 명확하게 판별할 수 있는 문장이나 식입니다.

    - *구분 예시*: "디미고는 경기도 안산시에 위치한다." (참인 명제) / "수학은 아주 재미있는 과목이다." (사람마다 느낌이 다르므로 명제가 아님).
- **진리값 (Truth Value)**
    - **뜻**: 어떤 명제가 참인지(1 또는 True) 혹은 거짓인지(0 또는 False)를 나타내는 논리적 결과값입니다.
- **논리 연산자 (Logical Operators)**
    - **AND (논리곱, ∧)**: 관련된 모든 조건이 **동시에 참**일 때만 전체 결과가 참이 됩니다.
    - **OR (논리합, ∨)**: 관련된 조건 중 **적어도 하나라도 참**이면 전체 결과가 참이 됩니다.
    - **NOT (논리부정, ~ 또는 ¬)**: 명제의 진리값을 반대로 뒤집습니다 (참 ➔ 거짓, 거짓 ➔ 참).
- **역 (Converse) / 이 (Inverse) / 대우 (Contrapositive)**
    - 조건 명제 **"P이면 Q이다 (P → Q)"**가 주어졌을 때:
        - **역 (Converse)**: 조건의 앞뒤 순서를 바꿈 (Q → P).
        - **이 (Inverse)**: 조건의 앞뒤를 각각 부정함 (~P → ~Q).
        - **대우 (Contrapositive)**: 순서도 바꾸고 둘 다 부정함 (~Q → ~P).
        - **★시험 핵심 법칙★**: **원래 명제가 참이면 그 대우 명제는 증명할 필요도 없이 무조건 100% 참**입니다! (원래 명제와 대우 명제는 항상 논리적 진리값이 일치합니다.)
- **삼단논법 (Syllogism)**
    - **뜻**: "P이면 Q이다"가 참이고, "Q이면 R이다"가 참일 때, 징검다리를 건너듯 연결하여 **"P이면 R이다"라는 새로운 참인 결론을 도출**하는 연역적 추론 규칙입니다.
- **모순 (Contradiction)**
    - **뜻**: 두 개 이상의 진술이나 전제가 서로 양립할 수 없어 논리적으로 충돌하는 상태입니다. "A는 항상 참말만 한다"는 가설에서 거짓된 결과가 도출되면 모순이 발생하여 최초 가설이 기각됩니다 (귀류법의 핵심).

3.2 이산수학 용어 (Discrete Mathematics)

- **이산수학 (Discrete Mathematics)**
    - **뜻**: '이산(Discrete)'이란 연속적이지 않고 똑똑 끊어져 낱개로 떨어진다는 뜻입니다. 실수나 미적분처럼 연속적인 값이 아니라, 0, 1, 2와 같은 정수, 집합, 경우

의 수, 그래프 등 컴퓨터 과학의 뼈대를 이루는 수학 체계입니다.

- **진법 변환 (Base Conversion)**
  - **뜻**: 숫자를 표현하는 자릿수 밑(Base)을 다른 체계로 상호 변환하는 기법입니다.
  - **10진수(Decimal)**: 인간이 일상에서 사용하는 0~9 체계.
  - **2진수(Binary)**: 컴퓨터 회로가 사용하는 0과 1 체계.
  - **16진수(Hexadecimal)**: 2진수 4자리를 `0~9, A, B, C, D, E, F` 한 글자로 압축하여 메모리 주소나 색상 코드 등에 사용하는 체계.
- **순열 (Permutation, nPr) 과 조합 (Combination, nCr)**
  - **순열(Permutation)**: 서로 다른 n개 중에서 r개를 골라 **순서를 고려하여 일렬로 나열**하는 경우의 수 (예: 4명 중 반장, 부반장 뽑기).
  - **조합(Combination)**: 서로 다른 n개 중에서 **순서에 상관없이** 모둠으로 r개를 선택만 하는 경우의 수 (예: 4명 중 대표 2명 뽑기, 악수 횟수 계산).
- **비둘기집 원리 (Pigeonhole Principle)**
  - **뜻**: 비둘기 n마리가 m개의 비둘기집에 들어가는데, 비둘기 수가 집의 수보다 많다면(n > m), **적어도 어느 한 비둘기집에는 2마리 이상의 비둘기가 반드시 들어있어야 한다**는 원리입니다. 최악의 시나리오를 따지는 논리 퍼즐의 기초가 됩니다.
- **그래프 (Graph) 와 트리 (Tree)**
  - **그래프(Graph)**: 데이터를 담고 있는 **점(정점/노드, Vertex/Node)**들과 이들을 서로 잇는 **선(간선, Edge)**들의 그물망 연결 구조입니다.
  - **트리(Tree)**: 그래프 중에서 빙글빙글 도는 순환선(사이클)이 전혀 없고, 하나의 뿌리 노드(Root)에서 아래로 가치 있게 뻗어나가는 계층적 자료구조입니다.
- **한붓그리기 (오일러 경로, Eulerian Path)**
  - **뜻**: 연필을 종이에서 떼지 않고, 모든 간선(선)을 정확히 한 번씩만 지나서 전체 도형을 그릴 수 있는 경로입니다. 연결된 선의 개수가 홀수 개인 정점이 0개이거나 정확히 2개일 때만 성립합니다.

3.3 자료구조 핵심 (Data Structures)

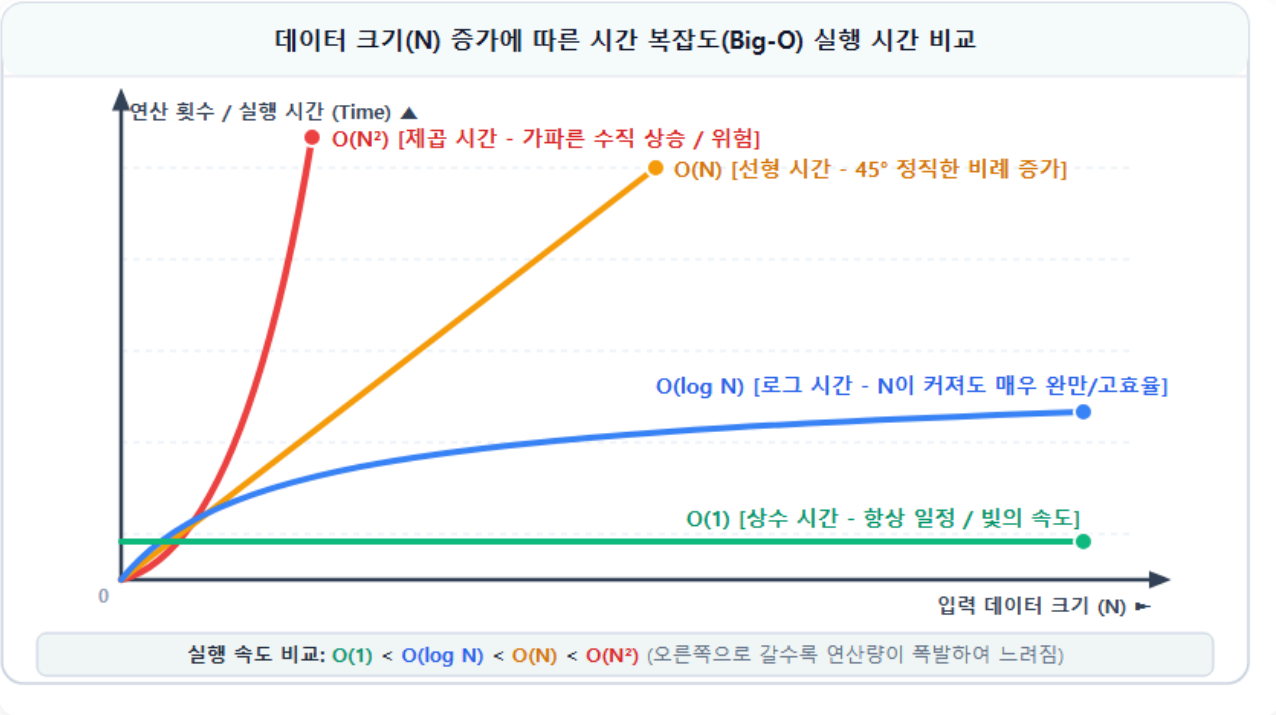
- **자료구조 (Data Structure)**
  - **뜻**: 컴퓨터 내부에서 대량의 데이터를 효율적으로 저장하고, 검색하고, 수정할 수 있도록 체계적으로 정리해 놓은 데이터의 구조와 틀입니다.
- **스택 (Stack: LIFO - 후입선출)**

    - **뜻**: 바닥이 막힌 상자처럼, **가장 나중에 들어간(Last In) 데이터가 가장 먼저 나오는(First Out)** 구조입니다.
    - *현실 예시*: 프링글스 감자칩 통, 책상 위에 쌓아 올린 책 더미, 웹 브라우저의 '뒤로 가기' 버튼, 텍스트 편집기의 '실행 취소(Ctrl+Z)'.
- **큐 (Queue: FIFO - 선입선출)**
    - **뜻**: 양쪽이 뚫린 파이프처럼, **가장 먼저 들어간(First In) 데이터가 가장 먼저 나오는(First Out)** 구조입니다.
    - *현실 예시*: 은행 번호표 창구 줄서기, 터널 통과, 프린터 인쇄 대기열, 맛집 대기 순번.
- **재귀 함수 (Recursive Function)**
    - **뜻**: 복잡하고 큰 문제를 더 작은 단위의 동일한 문제로 쪼개기 위해, **함수 내부에서 자기 자신을 다시 호출하여 풀어나가는 기법**입니다.
    - *주의점*: 함수가 끝없이 반복 실행되다가 컴퓨터 메모리가 터지는 것을 방지하기 위해, 반드시 재귀를 멈추고 반환하는 **기저 조건(Base Case)**이 필수적으로 포함되어야 합니다 (예: 팩토리얼 계산, 하노이의 탑).

3.4 알고리즘과 시간 복잡도 (Algorithms & Complexity)

- **알고리즘 (Algorithm)**
    - **뜻**: 어떤 문제를 해결하거나 목표한 결과를 얻기 위해 컴퓨터나 사람이 차례대로 밟아 나가야 하는 **명확하고 순서화된 유한한 단계별 절차의 모음**입니다 (9세기 아라비아의 대수학자 '알콰리즈미'의 이름에서 유래되었습니다).
    - **알고리즘이 왜 그렇게 중요할까요?**: 동일한 문제(예: 100만 명의 학생 데이터를 성적순으로 정리하기)를 컴퓨터에 시켜도, **어떤 알고리즘을 사용하느냐에 따라 0.01초 만에 끝날 수도 있고, 1시간이 걸려 컴퓨터가 멈춰버릴 수도 있기 때문**입니다. 최소한의 계산 횟수와 메모리로 최선의 결과를 내는 것이 알고리즘 연구의 핵심입니다.
- **순서도 (Flowchart)**
    - **뜻**: 알고리즘의 문제 해결 흐름과 판단 과정을 직사각형(명령), 마름모(조건 판단), 타원(시작/종료) 등 국제 표준 기호와 화살표를 이용해 시각적으로 일목요연하게 도식화한 그림입니다.
- **시간 복잡도 (Time Complexity) 와 빅오 표기법 (Big-O Notation)**
    - **뜻**: 알고리즘이 문제를 해결할 때, 입력 데이터의 개수(N)가 늘어남에 따라 **컴퓨터가 연산(계산)을 대략 몇 번이나 수행해야 하는지**를 수학적으로 나타낸 실행

효율성의 척도입니다.
    - **핵심 4단계 비교 (빠름 ➔ 느림: O(1) < O(log N) < O(N) < O(N²))**:
        1. **O(1) (상수 시간)**: 데이터 개수가 10개든 1억 개든 상관없이 단 **1번 만에** 결과를 즉시 찾아냄 (예: 사물함 번호 알고 바로 열기, 배열 인덱스 조회).
        2. **O(log N) (로그 시간)**: 탐색 대상을 매 단계 절반(1/2)씩 버려가며 찾아, 데이터 100만 개도 단 **20번의 계산**으로 찾아냄 (예: 스무고개 놀이, 이진 탐색).
        3. **O(N) (선형 시간)**: 데이터 개수 N에 비례하여 정직하게 하나씩 차례차례 확인 (예: 1중 반복문, 책장에 꽂힌 책을 1번부터 끝까지 훑는 선형 탐색).
        4. **O(N²) (제곱 시간)**: 데이터가 10배 늘어나면 연산량은 100배로 폭발하여 컴퓨터에 극심한 렉(Lag)을 유발함 (예: 2중 중첩 반복문, 버블 정렬).
- **시간 복잡도 곡선 그래프 (Time Complexity Graph)**:



3.5 정렬 및 6대 핵심 탐색 기법 (Sorting & Searching)

- **정렬 (Sorting)**
    - **뜻**: 뒤죽박죽 흩어져 있는 데이터들을 특정한 기준에 따라 크기 순서대로 오름차순(작은 값 ➔ 큰 값) 또는 내림차순(큰 값 ➔ 작은 값)으로 가지런히 정리하는 작업입니다 (예: 버블 정렬, 선택 정렬, 삽입 정렬 등).
- **탐색 (Search / Searching)**

- **뜻**: 대량의 데이터 뭉치 속에서 내가 찾고자 하는 특정 조건이나 목표값(Key)을 찾아내는 작업입니다.
- **종류별 6대 핵심 탐색 기법**:
  1. **선형 탐색 (Linear Search, 순차 탐색)**: 데이터의 처음부터 끝까지 순서대로 하나씩 비교하며 찾는 가장 단순하고 확실한 방법 (정렬되지 않은 데이터에 사용, O(N)).
  2. **이진 탐색 (Binary Search)**: **반드시 사전에 오름차순으로 정렬된 데이터**에서 중앙값을 비교하여, 찾는 대상이 왼쪽에 있는지 오른쪽에 있는지 판단 후 범위를 매번 절반씩 줄여나가는 고속 탐색 기법 (O(log N)).
  3. **완전 탐색 (브루트 포스, Brute Force)**: 가능한 모든 경우의 수를 하나도 빠짐없이 100% 전부 일일이 확인하여 정답을 찾아내는 무식하지만 가장 확실한 탐색 방법.
  4. **깊이 우선 탐색 (DFS, Depth-First Search)**: 갈림길이 나오면 한 방향을 정해 막다른 길(가장 깊은 바닥)에 다다를 때까지 끝까지 파고들며 탐색하는 방법 (스택 자료구조 또는 재귀 함수를 이용해 구현).
  5. **너비 우선 탐색 (BFS, Breadth-First Search)**: 시작 지점에서부터 거리가 가까운 이웃 노드(같은 층)부터 원을 그리듯 넓게 퍼져나가며 차례대로 탐색하는 방법 (큐 자료구조를 이용해 구현하며, **최단 경로 탐색**에 필수적).
  6. **백트래킹 (퇴각 검색, Backtracking)**: 정답을 찾아가다가 조건에 위배되는 막다른 길에 부딪히면, 더 이상 깊이 들어가지 않고 즉시 직전 갈림길로 되돌아와(Back) 다른 길을 모색하는 효율적인 가지치기(Pruning) 탐색 기법 (예: 미로 탈출, 스도쿠 퍼즐).

🛡️ 제4편: 정보 보안 및 최신 AI 테크 트렌드

4.1 정보 보안 및 암호 기술 (Security & Encryption)

- **해킹 (Hacking) 과 화이트 해커 (White Hacker)**
  - **해킹(크래킹)**: 정당한 권한 없이 다른 사람의 컴퓨터 시스템, 서버, 네트워크에 몰래 침입하여 데이터를 훔치거나 시스템을 파괴하는 사이버 범죄 행위입니다.
  - **화이트 해커(White Hacker)**: 나쁜 해커(블랙 해커)의 공격을 사전에 막기 위해, 시스템의 보안 취약점을 미리 찾아내어 튼튼한 방어벽을 구축해 주는 착한 보안 전문가입니다 (디미고 해킹방어과 학생들이 꿈꾸는 대표적인 진로입니다).
- **악성코드 (Malware) 와 랜섬웨어 (Ransomware)**

    - **악성코드(Malware)**: 컴퓨터나 스마트폰을 감염시켜 사용자를 괴롭히거나 정보를 빼돌리기 위해 악의적으로 제작된 모든 불법 소프트웨어의 총칭입니다.
    - **랜섬웨어(Ransomware)**: 시스템에 침투하여 피해자의 소중한 사진, 문서 파일들을 암호화하여 열지 못하게 '인질'로 잡고, 이를 풀어주는 대가로 금전(암호화폐)을 요구하는 악랄한 악성코드입니다.
- **방화벽 (Firewall)**
    - **뜻**: 인터넷 외부 망으로부터 들어오는 인가되지 않은 불법 침입이나 해킹 공격을 감시하고 차단하는 컴퓨터 네트워크의 **디지털 방어 성벽**입니다.
- **암호화 (Encryption) 기법의 핵심 분류**
    - **암호화**: 누군가 통신 내용을 중간에서 가로채더라도 내용을 알 수 없도록 외계어처럼 뒤섞는 기술이며, 복호화(Decryption)는 이를 원래의 문장으로 되돌리는 기술입니다.
    - **① 대칭키 암호화 (Symmetric Key Encryption)**: 암호화할 때 쓰는 열쇠와 복호화할 때 쓰는 **열쇠가 동일한** 방식입니다. 암호화 속도가 매우 빠르지만, 상대방에게 열쇠를 전달하는 과정에서 해커에게 탈취당할 위험이 있습니다.
    - **② 공개키 / 비대칭키 암호화 (Public Key Encryption)**: 자물쇠를 잠그는 열쇠(누구나 볼 수 있는 공개키)와 자물쇠를 여는 열쇠(주인만 가진 개인키)가 **서로 다른 쌍으로 동작**하는 방식입니다. 열쇠를 안전하게 보관할 수 있어 인터넷 뱅킹, 전자상거래 등에 핵심적으로 쓰입니다.
    - **③ 일방향 암호화 해시 (One-way Hash)**: 원본 데이터를 일정한 길이의 임의의 문자열로 완전히 뭉개버려, **그 누구도 원래의 데이터로 절대로 역산하여 되돌릴 수 없는** 기술입니다. 웹사이트 서버 관리자조차 사용자의 실제 비밀번호를 모르게 안전하게 보관할 때 사용합니다.

4.2 인공지능(AI) 핵심 용어 (Artificial Intelligence)

- **인공지능 (AI, Artificial Intelligence)**
    - **뜻**: 인간이 가진 시각, 청각, 언어 이해, 학습 및 문제 해결 능력을 컴퓨터 소프트웨어로 구현해 낸 포괄적인 기술 체계입니다.
- **머신러닝 (Machine Learning, 기계학습)**
    - **뜻**: AI를 개발하는 대표적인 방법론입니다. 개발자가 컴퓨터에게 규칙을 일일이 코딩해 주지 않고, **수많은 데이터(사진, 글 등)를 입력해 주면 컴퓨터가 스스로 데이터 속의 규칙과 패턴을 찾아내어 학습**하는 기술입니다.
- **딥러닝 (Deep Learning, 심층학습)**

    - **뜻**: 머신러닝의 하위 분야로, 인간의 **뇌 신경망(뉴런과 시냅스)** 구조를 본뜬 거대한 '인공신경망'을 층층이 깊게 쌓아 올려 스스로 고차원적 특징을 찾아내는 최첨단 학습 기술입니다 (이세돌 9단을 이긴 바둑 AI '알파고'가 대표적입니다).
- **생성형 AI (Generative AI)**
    - **뜻**: 기존 데이터를 단순히 분류하거나 예측하는 수준을 넘어, 사람의 프롬프트(명령어)에 반응하여 **글, 그림, 음악, 프로그래밍 소스 코드 등을 독창적으로 새롭게 창작해 내는 최신 인공지능**입니다 (예: ChatGPT, Midjourney, Claude).
- **LLM (거대 언어 모델, Large Language Model)**
    - **뜻**: 인간의 방대한 텍스트 데이터를 사전에 학습하여 사람처럼 자연스럽게 대화하고, 글을 요약하며, 코딩을 작성할 수 있는 **생성형 AI의 초대형 뇌 신경망 엔진**입니다.
- **AI 에이전트 (AI Agent)**
    - **뜻**: 질문에 단순히 수동적으로 답변하는 것을 넘어, 사용자가 부여한 목표를 달성하기 위해 **스스로 계획을 세우고, 인터넷 검색을 수행하며, 파일과 도구를 조작하여 복잡한 작업을 자율적으로 완수하는 능동적인 AI 비서**입니다.
- **AGI (범용 인공지능, Artificial General Intelligence)**
    - **뜻**: 특정 분야(체스, 번역 등)에만 국한되지 않고, 인간이 수행할 수 있는 모든 지적 작업을 사람과 동등하거나 그 이상으로 완벽하게 스스로 수행할 수 있는 차세대 최종형 인공지능입니다.
- **프롬프트 엔지니어링 (Prompt Engineering)**
    - **뜻**: 생성형 AI에게 원하는 최고 품질의 결과물을 얻어내기 위해, 지시어와 질문(프롬프트)을 논리적이고 정교하게 구성하고 다듬는 전문적인 명령 기술입니다.

4.3 디지털 신기술 (Emerging Tech)

- **빅데이터 (Big Data)**
    - **뜻**: 기존의 일반적인 데이터베이스 프로그램으로는 도저히 수집, 저장, 관리하기 어려울 만큼 **방대한 규모(Volume), 빠른 생성 속도(Velocity), 다양한 형태(Variety)**를 가진 대규모 데이터 군집입니다. 이를 분석하면 사람들의 선호도와 미래 트렌드를 정확히 예측할 수 있습니다.
- **사물인터넷 (IoT, Internet of Things)**
    - **뜻**: 냉장고, 세탁기, 시계, 자동차, 가로등 등 일상의 모든 사물에 센서와 네트워크 통신 기능을 탑재하여, 사람이 개입하지 않아도 사물들끼리 인터넷을 통해 정보를 실시간으로 주고받으며 원격 제어되는 기술입니다.

💼 제5편: e-비즈니스 및 경영·기술 융합

5.1 전자상거래 (E-Commerce)

- **전자상거래 (E-Commerce)**
  - **뜻**: 컴퓨터 네트워크와 인터넷을 매개로 하여 상품과 서비스를 사고파는 모든 디지털 상업 거래 행위를 뜻합니다.
  - **거래 주체별 3대 형태**:
    - **B2B (Business to Business)**: 기업과 기업 간의 원자재 및 부품 거래.
    - **B2C (Business to Consumer)**: 온라인 쇼핑몰(쿠팡, 네이버쇼핑 등)처럼 기업이 일반 소비자에게 상품을 판매하는 가장 일반적인 거래.
    - **C2C (Consumer to Consumer)**: 당근마켓, 중고나라처럼 개인과 개인 간에 이루어지는 중고 거래.

5.2 핀테크 (FinTech)

- **핀테크 (FinTech)**
  - **뜻**: **금융(Finance)**과 **기술(Technology)**을 결합한 합성어로, 모바일 송금, 간편 결제, 로보어드바이저 등 복잡한 전통 금융 서비스를 IT 기술을 통해 간소화하고 혁신한 서비스입니다 (예: 토스, 카카오페이, 애플페이).
  - **e-비즈니스에서의 핵심 가치**: 고객이 물건을 구매할 때 결제 절차가 복잡하면 구매를 포기하게 됩니다. 핀테크를 통해 지문 인식이나 터치 한 번으로 결제가 끝나게 만들어 줌으로써, **모든 온라인 비즈니스의 구매 전환율과 매출을 비약적으로 상승시키는 엔진 역할**을 합니다.

5.3 UI / UX (User Interface & User Experience)

- **UI (사용자 인터페이스, User Interface)**
  - **뜻**: 스마트폰 화면에서 사용자가 눈으로 직접 보고 손가락으로 누르는 버튼의 위치, 색상, 글꼴 크기, 메뉴 배치 등 **시각적인 디자인 구성 요소**를 뜻합니다.
- **UX (사용자 경험, User Experience)**
  - **뜻**: 사용자가 어떤 제품이나 앱 서비스를 처음 켜서 이용하고 결제하기까지의 전 과정에서 느끼는 **편리함, 감정, 만족감 등 총체적인 경험**을 말합니다. 아무리

화면(UI)이 예뻐도 버튼을 찾기 어렵고 결제가 번거롭다면 나쁜 UX가 됩니다.

5.4 플랫폼 비즈니스 (Platform Business)

- **플랫폼 비즈니스 (Platform Business)**
  - **뜻**: 자신이 직접 상품을 제조하거나 소유하지 않고, **공급자(판매자)와 수요자(구매자)가 서로 자유롭게 만나 거래할 수 있는 디지털 온라인 장터(기차역 승강장 같은 공간)를 마련해 주고 수수료나 광고 수익을 얻는 사업 모델**입니다.
  - *대표 예시*:
    - 배달의민족: 직접 치킨을 튀기지 않지만 식당과 손님을 연결.
    - 에어비앤비: 호텔 건물을 짓지 않지만 집주인과 여행객을 연결.
    - 우버: 직접 택시를 사지 않지만 운전기사와 승객을 연결.
  - **네트워크 효과 (Network Effect)**: 참여하는 사용자가 많아질수록 플랫폼의 가치가 눈덩이처럼 기하급수적으로 커져, 나중에는 다른 경쟁자가 시장에 발을 붙이기 힘들 정도로 막강한 독점적 경쟁력을 확보하게 됩니다. 현대 IT 기업들이 가장 열광하는 최고의 비즈니스 모델입니다.

🧩 제1편: 논리적 추론 (Logical Reasoning)

> **마치 탐정이 된 것처럼, 단서들을 조합해 진짜 정답을 찾아내는 훈련!**

디미고 소질·적성검사에서 '논리적 추론'은 방탈출 게임이나 추리 소설의 범인 찾기 게임과 비슷합니다. 주어진 조건(단서)들을 분석하여 모순이 없는 결론을 도출하는 능력을 평가합니다. 모든 핵심 개념에는 표준 영문 명칭을 병기하였으니 용어의 의미를 함께 익혀두세요.

---

1. 명제와 조건 (Proposition & Logic)

명제 (Proposition)란 참(True)인지 거짓(False)인지 객관적으로 판별할 수 있는 문장이나 식을 의미합니다. 컴퓨터 과학의 모든 알고리즘과 논리 회로는 이 참(1)과 거짓(0)의 판단에서 시작됩니다.

💡 필수 논리 연산자 (Logical Operators)

- **AND (논리곱, 그리고)**: 두 조건이 **모두 참**일 때만 결과가 참입니다.
  - *예:* "A는 중학생이고(AND), 여학생이다." ➔ 둘 다 만족해야 참.
- **OR (논리합, 또는)**: 두 조건 중 **하나 이상 참**이면 결과가 참입니다.
  - *예:* "A는 사과를 먹거나(OR), 배를 먹는다." ➔ 사과만 먹어도 참, 둘 다 먹어도 참. (둘 다 안 먹어야 거짓)
- **NOT (논리부정, ~이 아니다)**: 진리값을 반대로 뒤집습니다.
  - *예:* "비가 온다"의 부정 ➔ "비가 오지 않는다."

---

2. 역, 이, 대우와 삼단논법 (Core Logic Principles)

명제 문제에서 가장 많이 쓰이는 필수 원리 2가지입니다.

💡 1. 역 (Converse), 이 (Inverse), 대우 (Contrapositive)

조건 명제 **A이면 B이다 (A → B)**를 기본 명제라고 할 때:

구분	명칭	형태	원래 명제와의 관계
원래 명제	조건 명제	A → B	"비가 오면 우산을 쓴다." (참 가정)
역 (Converse)	순서 바꾸기	B → A	"우산을 쓰면 비가 온다." ➔ **항상 참은 아님** (햇빛 가리개일 수도 있음)
이 (Inverse)	둘 다 부정하기	~A → ~B	"비가 안 오면 우산을 안 쓴다." ➔ **항상 참은 아님**
대우 (Contrapositive)	바꾸고 둘 다 부정	~B → ~A	"우산을 쓰지 않았다면 비가 오지 않은 것이다." ➔ **무조건 100% 참!**

> 📌 **핵심 원칙: 원래 명제가 참이면, 그 대우(Contrapositive) 명제는 반드시 참이다!**
> 보기에서 원래 명제와 일치하는 문장을 찾으라고 하면 1초 만에 **대우**를 찾으시면 됩니다.

💡 2. 삼단논법 (Syllogism)

- **원리**: 두 명제 "A → B" 와 "B → C" 가 모두 참이라면, 중간의 B를 징검다리 삼아 **"A → C" 도 반드시 참**이 됩니다.
- **대우와의 융합**: 시험에서는 대우를 숨겨놓고 삼단논법을 묻습니다.
  - 조건 1: "컴퓨터를 좋아하는 학생은 수학을 잘한다." (A → B)
  - 조건 2: "논리적이지 않은 학생은 수학을 잘하지 못한다." (~C → ~B)
  - 조건 2의 **대우**를 구하면: "수학을 잘하는 학생은 논리적이다." (B → C)
  - 삼단논법 연결: A → B → C ➔ **"컴퓨터를 좋아하는 학생은 논리적이다." (A → C)** 라는 결론이 100% 참이 됩니다!

---

3. 참/거짓 추론 (Truth-teller & Liar Puzzle)

누가 거짓말을 하고 있고 누가 참말을 하는지 찾아내는 퍼즐입니다.

💡 풀이 비법: "모순(Contradiction) 찾기와 경우의 수 세팅"

1. **정반대 진술 찾기**: A가 "C가 범인이다", B가 "C는 범인이 아니다"라고 했다면, 둘 중 한 명은 반드시 거짓말쟁이이고 다른 한 명은 참말쟁이입니다.
2. **가정법(Hypothesis) 세우기**: "만약 A가 참말쟁이라면?" 하고 가정한 뒤, 나머지 조건들을 맞춰보다가 말이 안 되는 충돌(모순)이 생기면 가정을 뒤집고 반대 방향으로 확정합니다.

📝 실전 예제 1 (진실 게임)

> A, B, C 세 명의 용의자 중 한 명이 범인입니다. 한 명만 진실을 말하고 있을 때 범인은 누구일까요?
>
> - **A**: "제가 범인입니다."
> - **B**: "A는 범인이 아닙니다."
> - **C**: "저는 범인이 아닙니다."

✅ **풀이 과정:**

1. A와 B는 서로 정반대 진술을 하고 있으므로, 둘 중 한 명만 진실을 말하고 있습니다.
2. 진실을 말하는 사람이 단 1명이므로, C는 반드시 **거짓말**을 하고 있습니다.
3. C가 "저는 범인이 아닙니다"라고 거짓말을 했으므로, 실제 범인은 **C**입니다.

---

4. 조건 추리 및 공간/순서 배치 (Grid Deduction)

사람들을 줄 세우거나, 방에 배치하거나, 직업과 취미를 짝짓는 문제입니다. 머리로만 풀면 실수가 나오므로 반드시 **2차원 표(Grid)**나 직선을 그려 소거해 나갑니다.

📝 실전 예제 2 (순서 배치)

> 철수, 영희, 민수, 지민 4명이 100m 달리기를 했습니다.
>
> - [조건 1] 철수는 영희보다 먼저 결승선에 들어왔습니다.
> - [조건 2] 민수는 1등도 아니고 4등도 아닙니다.

> - [조건 3] 지민이는 철수 바로 다음(연속해서)에 들어왔습니다.
>   1등부터 4등까지의 순서를 구하시오.

✅ **풀이 과정:**

1. 4개의 등수 칸을 준비합니다: [1등] - [2등] - [3등] - [4등]
2. 조건 3에 의해 `(철수-지민)`은 묶음으로 인접해야 합니다.
3. 만약 (철수-지민)이 2등-3등이라면, 1등 자리에 올 수 있는 사람이 없습니다 (민수는 1등 불가, 영희는 철수보다 뒤여야 하므로 불가).
4. 따라서 `(철수-지민)`은 무조건 **1등-2등**입니다.
5. 남은 3등, 4등에 민수와 영희를 배치해야 하는데, 조건 2에서 민수는 4등이 아니므로 3등입니다. 따라서 영희는 4등입니다.
   정답: 1등 철수 ➔ 2등 지민 ➔ 3등 민수 ➔ 4등 영희

✍️ 단원 마무리 실전 연습 문제

[문제 1]

다음 두 문장이 모두 참일 때, 반드시 참이라고 볼 수 있는 결론은?

- 전제 1: "책을 많이 읽는 사람은 어휘력이 좋다."
- 전제 2: "생각이 깊지 않은 사람은 어휘력이 좋지 않다."

① 어휘력이 좋은 사람은 책을 많이 읽는다.
② 생각이 깊은 사람은 책을 많이 읽는다.
③ 책을 많이 읽는 사람은 생각이 깊다.
④ 생각이 깊지 않은 사람은 책을 많이 읽는다.

[문제 2]

A, B, C, D 4명의 학생이 시험을 치렀습니다. 1등부터 4등까지의 성적에 대해 다음과 같은 진술이 있습니다.

- "A는 B보다 성적이 높다."
- "C는 D보다 성적이 높다."

- "B는 C보다 성적이 높다."
  이때 3등을 한 학생은 누구입니까? (단, 동점자는 없습니다.)

🎯 정답 및 해설

[문제 1 정답] ③

- [해설]
  - 전제 1: 책을 많이 읽음(A) ➔ 어휘력이 좋음(B)
  - 전제 2: 생각이 깊지 않음(~C) ➔ 어휘력이 좋지 않음(~B)
  - 전제 2의 **대우**를 구하면: 어휘력이 좋음(B) ➔ 생각이 깊음(C)
  - 삼단논법 적용: A → B 이고 B → C 이므로, A → C 즉 **책을 많이 읽는 사람은 생각이 깊다**가 반드시 참이 됩니다.

[문제 2 정답] C

- [해설]
  - 조건들을 부등호로 표현합니다:
    - A > B
    - C > D
    - B > C
  - 세 부등식을 연결하면: A > B > C > D
  - 따라서 순서는 1등 A, 2등 B, 3등 C, 4등 D 입니다. 3등은 **C**입니다.

🔢 제2편: 이산수학 (Discrete Mathematics)

> **컴퓨터가 세상을 세고, 계산하고, 연결하는 디지털 수학의 세계!**

이산수학(Discrete Mathematics)은 연속적인 실수나 미적분이 아니라, 1, 2, 3처럼 딱딱 끊어지는 정수, 경우의 수, 논리, 연결 관계(네트워크)를 연구하는 컴퓨터 과학의 근본 학문입니다. 모든 주요 용어에는 표준 영문 명칭을 함께 표기했습니다.

---

1. 진법 변환과 2진수 연산 (Base Conversion)

인간은 손가락이 10개여서 0부터 9까지를 사용하는 **10진법(Decimal)**을 주로 씁니다. 하지만 컴퓨터는 전기가 흐르지 않음(0)과 흐름(1)만을 인식하므로 **2진법(Binary)**을 사용합니다. 나아가 데이터를 압축해 효율적으로 표기하기 위해 **8진법(Octal)**과 **16진법(Hexadecimal)**을 함께 활용합니다.

💡 1. 10진수를 2진수로 변환 (나눗셈 활용)

10진수를 2로 계속 나누어 나오는 **나머지**를 밑에서부터 거꾸로 적습니다.

[예제] 10진수 13을 2진수로 변환

1. 13 ÷ 2 = 6 ... 나머지 **1**
2. 6 ÷ 2 = 3 ... 나머지 **0**
3. 3 ÷ 2 = 1 ... 나머지 **1**
4. 1 ÷ 2 = 0 ... 나머지 **1**
   ➔ 밑(마지막 몫)에서부터 거꾸로 읽으면: `1101(2)`

💡 2. 2진수를 10진수로 변환 (자릿값 활용)

오른쪽 끝(일의 자리)부터 자릿값(2^0=1, 2^1=2, 2^2=4, 2^3=8, 2^4=16, ...)을 곱해 모두 더합니다.

[예제] 2진수 `1101(2)`를 10진수로 변환

- (1 × 8) + (1 × 4) + (0 × 2) + (1 × 1) = 8 + 4 + 0 + 1 = **13(10)**

💡 3. 16진수(Hexadecimal)와 2진수의 상호 변환

16진수는 0부터 15까지 16개의 기호를 사용합니다. 10부터 15까지는 알파벳 **A(10), B(11), C(12), D(13), E(14), F(15)**로 표기합니다.

- **핵심 규칙**: 2진수 **4자리**는 16진수 **1자리**와 정확히 일치합니다 ($2^4 = 16$).
- **[예제] 2진수 `10110110(2)`를 16진수로 변환**
  - 뒤에서부터 4자리씩 분할: `1011` / `0110`
  - 앞자리 `1011(2)` = 8 + 0 + 2 + 1 = 11 ➔ 16진수 **B**
  - 뒷자리 `0110(2)` = 0 + 4 + 2 + 0 = 6 ➔ 16진수 **6**
  - 결과: **`B6(16)`**

💡 4. 음수 표현과 2의 보수 (2's Complement)

컴퓨터는 마이너스(`-`) 기호 회로를 따로 만들지 않고, 덧셈 회로 하나로 뺄셈까지 처리하기 위해 **2의 보수(2's Complement)** 방식을 씁니다.

1. **1의 보수 (1's Complement)**: 모든 비트를 반전(NOT)시킵니다 (0은 1로, 1은 0으로).
2. **2의 보수 (2's Complement)**: 1의 보수 결과에 **1을 더합니다**.

> **[예제] 8비트에서 -5 표현하기**
>
> 1. +5의 2진수: `0000 0101`
> 2. 1의 보수 취하기: `1111 1010`
> 3. 1 더하기: `1111 1010` + `1` = **`1111 1011` (-5)**
>
> *(맨 앞자리 비트가 1이면 음수, 0이면 양수를 의미합니다.)*

---

2. 경우의 수와 조합론 (Combinatorics)

💡 1. 합의 법칙과 곱의 법칙

- **합의 법칙 (Rule of Sum)**: 사건 A 또는 사건 B가 **따로(동시에 일어나지 않게)** 발생할 때 ➔ 각 경우의 수를 **더합니다 (+)**.

    - *예:* 버스 3가지 노선 중 하나를 타거나, 지하철 2가지 노선 중 하나를 타는 경우 = 3 + 2 = 5가지.
- **곱의 법칙 (Rule of Product)**: 사건 A와 사건 B가 **잇달아(동시에)** 발생할 때 ➔ 각 경우의 수를 **곱합니다 (×)**.
    - *예:* 상의 4벌과 하의 3벌을 매칭하여 입는 코디의 수 = 4 × 3 = 12가지.

💡 2. 순열 (Permutation) vs 조합 (Combination)

두 개념의 결정적 차이는 **순서가 존재하는가?**입니다.

구분	순열 (Permutation, nPr)	조합 (Combination, nCr)
의미	서로 다른 n개 중 r개를 택해 **순서를 따져 일렬로 나열**	서로 다른 n개 중 순서와 무관하게 **r개를 선택만 함**
대표 상황	반장 1명과 부반장 1명 선출, 이어달리기 순서	대표 2명 선출, 모둠원 뽑기, **사람들끼리 악수하기**
계산 공식	nPr = n × (n-1) × … × (n-r+1)	nCr = nPr ÷ r! = (n × (n-1) × …) ÷ (r × … × 1)
예시 (4명 중 2명)	4P2 = 4 × 3 = **12가지**	4C2 = (4 × 3) ÷ (2 × 1) = **6가지**

💡 3. 비둘기집 원리 (Pigeonhole Principle)

- **원리**: (N + 1)마리의 비둘기가 N개의 비둘기집에 들어간다면, **적어도 한 개의 집에는 2마리 이상의 비둘기가 들어있어야 합니다.**
- **응용 (최악의 경우 추론)**:
    - *상황:* 주머니에 빨간 양말 10짝, 파란 양말 10짝이 섞여 있을 때, 눈을 감고 양말을 꺼내어 **반드시 한 켤레(같은 색 2짝)**를 맞추려면 최소 몇 짝을 꺼내야 할까요?
    - *풀이:* 운이 가장 나쁜 경우(최악)를 가정합니다. 처음에 빨간 양말 1짝, 다음에 파란 양말 1짝을 뽑아 색이 갈라졌다고 해도, **3번째 양말**을 꺼내면 빨간색이든 파란색이든 반드시 기존 양말과 짝을 이루게 됩니다. 따라서 최소 **3짝**을 꺼내야 합니다.

---

3. 그래프 이론 (Graph Theory)

그래프(Graph)는 데이터를 나타내는 **점(정점/노드, Vertex/Node)**과 이들을 연결하는 **선(간선, Edge)**의 집합입니다.

💡 1. 악수 정리 (Handshaking Lemma)

모든 간선(선)은 항상 양 끝에 2개의 정점을 연결합니다. 따라서 다음과 같은 절대 법칙이 성립합니다.

> **모든 점의 연결선 수(차수의 총합) = 간선(선분)의 수 × 2**

- *예:* 점이 6개 있고 각 점에서 선이 4개씩 뻗어 나온다면, 연결선 수의 총합은 6 × 4 = 24입니다. 선분은 2개 점을 연결하므로 총 선분의 수는 24 ÷ 2 = **12개**입니다.

💡 2. 한붓그리기와 오일러 경로 (Eulerian Path)

연필을 떼지 않고 모든 간선을 한 번씩만 지나가는 경로를 오일러 경로(Eulerian Path)라고 합니다.

- **짝수점(차수가 짝수)**: 선을 타고 들어왔다가 다른 선으로 나갈 수 있는 중간 경유점.
- **홀수점(차수가 홀수)**: 들어오면 나갈 수 없으므로 반드시 **출발점** 또는 **도착점**이 되어야 하는 점.
- **한붓그리기 가능 조건**:
  1. 홀수점이 **0개**일 때 (어느 점에서 출발해도 다시 출발점으로 돌아오는 오일러 회로 완성).
  2. 홀수점이 **정확히 2개**일 때 (한 홀수점에서 출발하여 다른 홀수점에서 끝남).
     (홀수점이 4개 이상이면 한붓그리기는 절대 불가능합니다.)

💡 3. 트리 (Tree)의 성질

트리는 사이클(순환선)이 없는 연결 그래프입니다.

- **핵심 공식**: 정점(Vertex)의 개수가 V개라면, 트리의 간선(Edge) 개수 E는 항상 **(V - 1)개**입니다.
  - 정점이 10개인 트리의 선분 개수는 계산할 필요도 없이 10 - 1 = **9개**입니다.

✍️ 단원 마무리 실전 연습 문제

[문제 1]

10진수 숫자 45를 2진수로 올바르게 나타낸 것은?

① `101101(2)`

② `101110(2)`

③ `110101(2)`

④ `111001(2)`

[문제 2]

어느 반에 8명의 학생이 있습니다. 방과 후 모든 학생이 서로 빠짐없이 한 번씩 악수를 나누었습니다. 이 반에서 이루어진 총 악수 횟수는 몇 번입니까?

[문제 3]

어떤 그래프에 5개의 정점이 있습니다. 각 정점에 연결된 간선의 개수가 각각 2개, 3개, 3개, 4개, 4개입니다. 이 그래프에 존재하는 간선(선분)의 총개수는 몇 개입니까?

🎯 정답 및 해설

[문제 1 정답] ①

- [해설]
  - 45를 2로 나눈 나머지들을 역순으로 나열:
  - $45 = 32 + 8 + 4 + 1 = 2^5 + 2^3 + 2^2 + 2^0$
  - 자릿값 배치: 32(1), 16(0), 8(1), 4(1), 2(0), 1(1) ➔ **`101101(2)`**

[문제 2 정답] 28번

- [해설]
  - 8명 중 순서에 상관없이 2명을 선택하는 조합(nCr) 문제입니다.
  - $8C2 = (8 \times 7) \div (2 \times 1) =$ **28번**.

[문제 3 정답] 8개

- [해설]
  - 악수 정리 공식: 모든 정점의 차수의 총합 = 간선 수 × 2
  - 차수의 총합 = 2 + 3 + 3 + 4 + 4 = 16
  - 간선 수 = 16 ÷ 2 = **8개**.

💻 제3편: 컴퓨팅 사고력 (Computational Thinking)

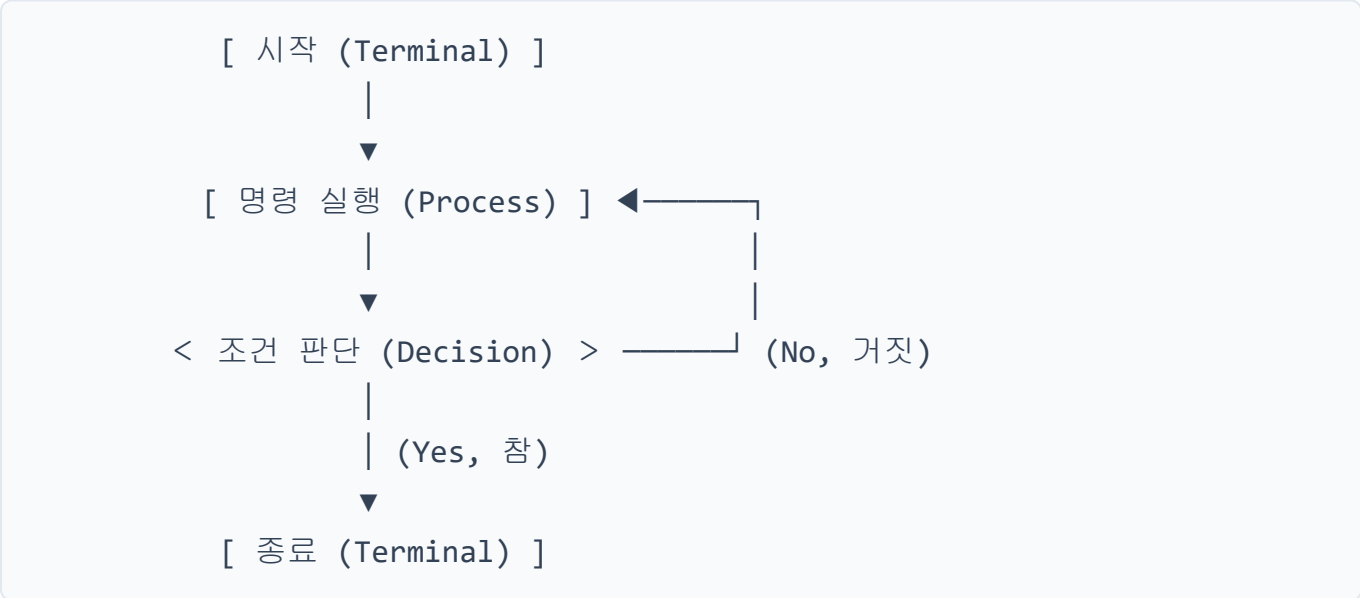
> **문제를 컴퓨터가 이해할 수 있도록 논리적으로 분해하고 절차화하는 능력!**

컴퓨팅 사고력(Computational Thinking)은 특정 프로그래밍 언어의 문법을 외우는 것이 아닙니다. **복잡한 문제를 작은 단위로 쪼개고(분해), 핵심 패턴을 찾아내며(추상화), 이를 해결하기 위한 명확한 절차(알고리즘)를 설계하는 문제 해결력**입니다. 모든 핵심 개념에는 표준 영문 명칭을 병기했습니다.

---

1. 순서도 (Flowchart) 읽기와 분석

순서도(Flowchart)는 문제 해결의 알고리즘 절차를 표준 기호와 화살표로 시각화한 도표입니다.



💡 주요 표준 기호

- **타원 (단말, Terminal)**: 프로그램이나 알고리즘의 **시작(Start)**과 **끝(End)**을 나타냅니다.
- **직사각형 (처리, Process)**: 연산, 계산, 변수 대입 등의 구체적인 동작을 실행합니다 (예: `X = X + 1`, `1칸 전진`).

- **마름모 (판단/조건, Decision)**: 참(Yes/True) 또는 거짓(No/False)에 따라 실행 흐름이 갈라지는 분기점입니다 (예: `X > 10?`, `앞에 장애물이 있는가?`).
- **평행사변형 (입출력, Input/Output)**: 외부로부터 데이터를 입력받거나 화면에 출력(PRINT)합니다.

---

2. 변수와 제어 구조 추적 (Tracing Algorithm)

알고리즘 문제의 90%는 코드가 실행됨에 따라 메모리(변수)의 값이 어떻게 변하는지 **손으로 표를 그려 추적(Trace)**하는 능력에서 결정됩니다.

💡 1. 알고리즘의 3대 기본 제어 구조 (Control Structures)

모든 컴퓨터 프로그램과 알고리즘은 단 3가지 기본 제어 구조의 조합으로 설계됩니다.

- **순차 구조 (Sequence)**: 명령어들을 위에서 아래로 정해진 순서에 따라 차례대로 하나씩 실행합니다.
- **선택 / 분기 구조 (Selection / Branching)**: 주어진 조건식의 참/거짓 판단 결과에 따라 실행 경로를 둘 이상으로 나눕니다 (`IF - ELSE`).
- **반복 구조 (Iteration / Loop)**: 정해진 조건이 참인 동안 특정한 명령 블록을 되풀이하여 실행합니다 (`WHILE`, `FOR`).

💡 2. 변수(Variable)와 값 교환 (SWAP)

변수는 값을 보관하는 이름표가 붙은 상자입니다.

- `A = B` 라는 표현은 수학의 "A와 B가 같다"가 아니라, **오른쪽 B의 값을 왼쪽 A 상자에 복사하여 덮어쓴다(대입, Assignment)**는 뜻입니다.
- **SWAP 알고리즘 (두 변수의 값 맞바꾸기)**:
  임시 보관 상자 `TEMP`를 활용하여 두 컵의 음료를 섞이지 않게 교환하는 방식입니다.

```
TEMP = X   // 1. X의 값을 임시 상자에 보관
X = Y      // 2. Y의 값을 X에 덮어씀
Y = TEMP   // 3. 임시 상자의 원래 X값을 Y에 넣음
```

💡 3. 의사코드(슈도코드, Pseudo-code)와 반복문 추적

- **조건문 (`IF - ELSE`)**: 조건이 참일 때와 거짓일 때 서로 다른 경로를 실행합니다.
- **반복문 (`WHILE`, `FOR`)**: 정해진 조건이 만족되는 동안 내부 명령 블록을 되풀이하여 실행합니다.

[실전 추적 예제]

```
SUM = 0
I = 1
WHILE (I <= 4) {
    SUM = SUM + I
    I = I + 1
}
PRINT(SUM)
```

- **추적 표 (Trace Table)**:

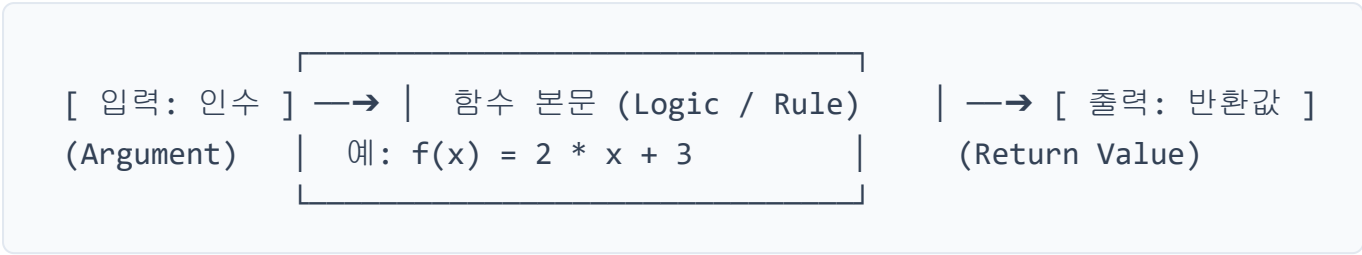
반복 회차	루프 진입 전 상태	루프 내부 실행 (`SUM=SUM+I`, `I=I+1`)	다음 루프 조건 검사 (`I <= 4`)
1회차	SUM=0, I=1	SUM = 0+1 = **1**, I = 1+1 = **2**	2 <= 4 (참 ➔ 계속)
2회차	SUM=1, I=2	SUM = 1+2 = **3**, I = 2+1 = **3**	3 <= 4 (참 ➔ 계속)
3회차	SUM=3, I=3	SUM = 3+3 = **6**, I = 3+1 = **4**	4 <= 4 (참 ➔ 계속)
4회차	SUM=6, I=4	SUM = 6+4 = **10**, I = 4+1 = **5**	5 <= 4 (**거짓 ➔ 루프 종료**)

- **결과**: 루프 탈출 후 `PRINT(SUM)` 실행 시 화면에 `10`이 출력됩니다.

---

3. 함수와 모듈화 사고 (Functions & Modularity)

프로그래밍과 컴퓨팅 사고력에서 **함수(Function)**는 가장 강력한 문제 해결 도구입니다. 복잡한 문제를 한 번에 해결하려고 하지 않고, 독립적인 작은 단위의 기능(부품)으로 쪼개어 해결하는 **모듈화(Modularity)**의 핵심입니다.



💡 1. 함수의 3대 핵심 구성 요소

1. **함수 이름 (Function Name)**: 이 함수가 어떤 기능을 수행하는지 명확히 나타내는 식별자입니다 (예: `calculateTax`, `findMax`).
2. **매개변수 (Parameter / 인자 Argument)**: 외부에서 함수 내부로 작업을 위해 건네주는 **입력 재료(데이터)**입니다. 매개변수가 없는 함수도 있습니다.
3. **반환값 (Return Value)**: 함수 내부의 연산이 끝난 후, 함수를 호출했던 원래 자리로 건네주는 **최종 결과물(출력)**입니다.

```
// 함수 정의 (Definition)
FUNCTION addNumbers(a, b) {     // a, b는 매개변수 (Parameter)
    sum = a + b                 // 내부 연산 처리
    RETURN sum                  // 최종 계산 결과를 반환
}

// 함수 호출 (Call)
total = addNumbers(10, 20)      // 10, 20은 전달인자 (Argument)
PRINT(total)                    // 화면에 30 출력
```

💡 2. 디미고 빈출 유형: 블랙박스 규칙 추론 (변환 상자)

시험에서는 내부 규칙이 가려진 **변환 상자(블랙박스 함수)**에 숫자를 넣었을 때의 입출력 표를 주고, 숨겨진 함수 규칙 $f(x)$를 찾아내어 물음표의 값을 구하라는 문제가 단골로 출제됩니다.

입력 (x)	출력 (y)	규칙 분석 (힌트 찾기)
2	**5**	2배 하면 4 ➔ 1이 부족함 (+1)
4	**9**	4배 하면 8 ➔ 1이 부족함 (+1)
6	**13**	6배 하면 12 ➔ 1이 부족함 (+1)

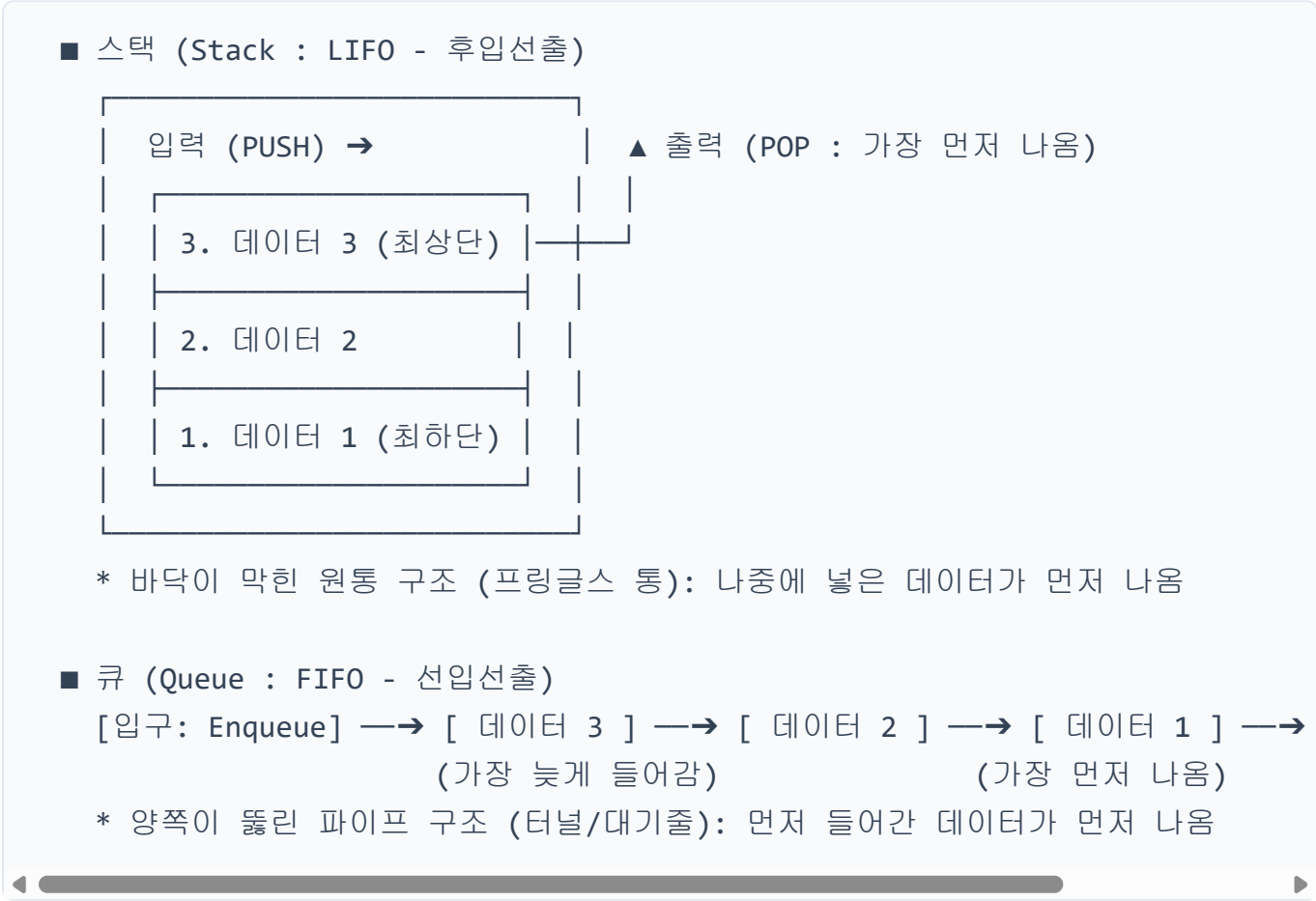
입력 (x)	출력 (y)	규칙 분석 (힌트 찾기)
9	**?**	$f(x) = 2x + 1$ 규칙 성립!

- **풀이 비법**:
  1. **변화율(기울기) 관찰**: 입력이 2 커질 때 출력이 4 커졌으므로, 입력값에 **2를 곱하는 규칙($2x$)**이 기본 뼈대입니다.
  2. **상수 보정**: $2 \times 2 = 4$인데 출력이 5이므로, 뒤에 $+1$을 더해줍니다.
  3. **검증 및 정답 도출**: $f(x) = 2x + 1$ 이므로, 9를 넣었을 때의 결과는 $2 \times 9 + 1 = \mathbf{19}$ 입니다.

---

4. 핵심 자료구조: 스택과 큐 (Stack & Queue)

컴퓨터가 데이터를 어떤 순서로 저장하고 꺼낼 것인지에 대한 기본 약속입니다.



💡 1. 스택 (Stack)

- **LIFO (Last In First Out, 후입선출)**: 가장 마지막에 들어간 데이터가 가장 먼저 나옵니다.
- **핵심 연산**: 데이터를 넣는 **Push**, 맨 위 데이터를 꺼내는 **Pop**.
- **실제 활용**: 웹 브라우저의 '뒤로가기', 문서 편집기의 '실행 취소(Ctrl+Z)', 수식의 괄호 검사.

💡 2. 큐 (Queue)

- **FIFO (First In First Out, 선입선출)**: 가장 먼저 들어간 데이터가 가장 먼저 나옵니다.
- **핵심 연산**: 줄 뒤에 데이터를 넣는 **Enqueue**, 맨 앞 데이터를 꺼내는 **Dequeue**.
- **실제 활용**: 은행 번호표 대기열, 프린터 인쇄 대기 목록, 맛집 웨이팅 시스템.

---

5. 심화 컴퓨팅 사고: 재귀 함수와 격자 탐색 (Recursion & Grid Simulation)

💡 1. 재귀 함수 (Recursive Function)

- **개념**: 함수가 어떤 복잡한 문제를 해결하는 과정에서 **'자기 자신과 똑같은 함수'를 더 작은 크기의 입력값으로 다시 호출**하는 기법입니다.
- **필수 조건 (절대 법칙)**: 무한 루프에 빠져 컴퓨터가 멈추지(Stack Overflow) 않도록 종료 조건을 명시한 **기저 조건(Base Case)**이 반드시 첫 줄에 있어야 합니다.
- **대표 예시 (팩토리얼, Factorial)**:
  N! = N × (N-1)! (단, 0! = 1, 1! = 1)

```
FUNCTION FACTORIAL(N) {
    IF (N <= 1) {
        RETURN 1           // 기저 조건 (Base Case): 1 이하이면 더 이상 호
    } ELSE {
        RETURN N * FACTORIAL(N - 1)  // 자기 자신을 더 작은 값(N-1)으로 자
    }
}
```

- **호출 단계 추적**:
    1. `FACTORIAL(3)` 호출 ➔ `3 * FACTORIAL(2)` 대기
    2. `FACTORIAL(2)` 호출 ➔ `2 * FACTORIAL(1)` 대기
    3. `FACTORIAL(1)` 호출 ➔ 기저 조건 만족! `1` 반환!
    4. 역순으로 곱셈 완성: $3 \times (2 \times 1) = 6$.

💡 2. 2차원 격자 좌표 이동 (2D Grid Simulation)

로봇이나 캐릭터가 바둑판 격자 위를 이동할 때, 방향 전환과 좌표 변화를 논리적으로 계산합니다.

- **방향 벡터 (북, 동, 남, 서)**:
    - 북쪽(위): Y 좌표 1 증가 (X, Y+1)
    - 동쪽(오른쪽): X 좌표 1 증가 (X+1, Y)
    - 남쪽(아래): Y 좌표 1 감소 (X, Y-1)
    - 서쪽(왼쪽): X 좌표 1 감소 (X-1, Y)
- 오른쪽 회전(우회전 90도): 북 ➔ 동 ➔ 남 ➔ 서 ➔ 북 순환.
- 왼쪽 회전(좌회전 90도): 북 ➔ 서 ➔ 남 ➔ 동 ➔ 북 순환.

---

6. 알고리즘 및 핵심 탐색 기법 (Algorithms & Search Techniques)

컴퓨터 과학에서 문제를 효율적으로 해결하기 위해 사용하는 알고리즘의 기초와 핵심 탐색 기법들입니다.

💡 0. 알고리즘(Algorithm)이란 무엇인가? (개념과 본질)

- **정의**: 어떤 문제를 해결하거나 목표한 결과를 얻기 위해 정해진 **명확하고 순서화된 단계별 해결 절차나 규칙의 모음**입니다.
- **어원**: 9세기 페르시아(아라비아)의 위대한 수학자 **알콰리즈미(al-Khwarizmi)**의 이름에서 유래되었습니다.
- **실생활 비유**:
    - **요리책의 레시피**: 라면을 끓일 때 "물 550ml 끓이기 ➔ 스프 넣기 ➔ 면 넣기 ➔ 4분간 끓이기"처럼 누구나 순서대로 똑같이 따라 하면 맛있는 라면이 완성되는 정해진 절차.

    - **내비게이션 길 안내**: 출발지부터 목적지까지 갈림길마다 "100m 앞 우회전 ➔ 지하차도 진입"처럼 운전자가 따라야 할 최적의 행동 순서.
- **알고리즘이 왜 그토록 중요할까요?**:
  컴퓨터에게 100만 명의 학생 명단을 이름순으로 정렬하라고 시켰을 때, **어떤 알고리즘을 쓰느냐에 따라 0.01초 만에 끝날 수도 있고, 1시간이 걸려 컴퓨터가 멈춰버릴 수도 있기 때문**입니다. 최소한의 계산(시간)과 최소한의 메모리(공간)로 최고의 결과를 내는 효율적인 절차를 설계하는 것이 알고리즘의 본질입니다.

💡 1. 시간 복잡도 (Time Complexity) 와 빅오 표기법 (Big-O Notation)

알고리즘의 성능을 평가할 때 "내 컴퓨터에서는 3초 걸리는데, 네 컴퓨터에서는 0.1초 걸리네?" 처럼 '초(Seconds)'로 재면 컴퓨터 성능에 따라 결과가 달라집니다. 그래서 컴퓨터 과학에서는 하드웨어 성능과 상관없이 **입력 데이터의 개수 *N*이 커질 때, 컴퓨터가 연산(계산)을 몇 번이나 수행해야 하는가?**를 수학적으로 표기합니다. 이를 **시간 복잡도(Time Complexity)**라고 하며, 가장 운이 나쁜 최악의 경우를 기준으로 표기하는 방법을 **빅오 표기법 (Big-O Notation)**이라고 합니다.

시험에 반드시 나오는 4대 빅오 표기법을 가장 직관적인 일상생활 비유로 완벽히 정리해 봅시다!

⚡ 1. O(1) [상수 시간, Constant Time]: "빛의 속도! 데이터가 100만 개라도 딱 1번에 끝!"

- **직관적 비유**: 내 사물함 열쇠에 '25번'이라고 적혀 있습니다. 학교에 사물함이 10개 있든, 10만 개 있든 상관없이, 1번부터 찾지 않고 그냥 곧장 25번 사물함 앞으로 걸어가서 **단 1번에** 엽니다!
- **코드 패턴**: 인덱스를 알고 있는 배열 값 조회 (`Array[25]`), 두 숫자의 단순 덧셈 (`A + B`), 스택의 `PUSH` / `POP`.
- **특징**: 데이터 N의 크기와 상관없이 실행 횟수가 항상 1번(일정함)인 가장 이상적인 속도입니다.

🚀 2. O(log N) [로그 시간, Logarithmic Time]: "스무고개의 마법! 데이터가 2배로 늘어나도 질문은 딱 1번만 늘어남!"

- **직관적 비유**: 1부터 100 사이의 숫자를 맞추는 '업/다운 게임(스무고개)'을 생각해보세요. 첫 질문으로 무조건 가운데인 **50보다 큽니까?**를 묻습니다. 이 단 한 번의 질문

으로 50개의 오답 후보가 한순간에 날아갑니다! 1,000페이지짜리 두꺼운 영어 사전에서 단어를 찾을 때 정가운데를 펴서 찾는 것도 같은 원리입니다.

- **놀라운 효율성**: 데이터가 1,000개일 때는 단 10번(2^{10} = 1,024), 데이터가 **100만 개(N=1,000,000)라도 단 20번($2^{20} \approx 1{,}000{,}000$) 만에** 원하는 답을 찾아냅니다!
- **코드 패턴**: 매 반복마다 탐색 범위가 절반(/2)씩 줄어드는 반복문.

```
WHILE (N > 1) {
    N = N / 2   // 매 단계마다 N이 반토막 남! ➔ O(log N)
}
```

- **대표 알고리즘**: **이진 탐색 (Binary Search)**.

🚗 3. O(N) [선형 시간, Linear Time]: "정직한 성실파! 데이터 개수만큼 차례대로 확인하기"

- **직관적 비유**: 바닥에 떨어진 에어팟 한 짝을 찾기 위해 방바닥을 왼쪽 구석부터 오른쪽 끝까지 한 걸음씩 훑어보는 것. 책 100권이 책장에 꽂혀 있을 때, 내가 찾는 책이 나올 때까지 맨 앞부터 한 권씩 차례대로 꺼내보는 것과 같습니다.
- **특징**: 데이터가 10개면 10번, 데이터가 1만 개면 1만 번 계산해야 합니다. 데이터 크기 N과 실행 시간이 정직하게 1:1로 비례합니다.
- **코드 패턴**: 1부터 N까지 차례대로 순회하는 **1중 반복문(단일 루프)**.

```
FOR (I = 1 to N) {
    SUM = SUM + I  // N번만큼 정직하게 실행 ➔ O(N)
}
```

- **대표 알고리즘**: **선형 탐색 (Linear Search)**, 배열의 최댓값/최솟값 찾기, 1부터 N까지 누적 합 구하기.

🐢 4. O(N^2) [제곱 시간, Quadratic Time]: "위험한 거북이! 데이터가 늘어나면 컴퓨터가 멈춘다!"

- **직관적 비유**: 우리 반 학생 N명이 서로 빠짐없이 한 번씩 키를 재어 비교하거나, 악수를 나누는 상황입니다. 학생이 10명이면 약 100번(10^2) 비교하면 되지만, 전교생 1,000명이면 1,000 × 1,000 = 1,000,000 (백만 번!) 비교해야 합니다. 데이터가 조금만 커져도 연산량이 폭발하여 컴퓨터가 렉(Lag)에 걸립니다.

- **코드 패턴**: 반복문 안에 또 다른 반복문이 들어있는 **2중첩 반복문 (Nested Loop)**.

```
FOR (I = 1 to N) {
    FOR (J = 1 to N) {
        // N * N 번 실행됨! ➔ O(N²)
    }
}
```

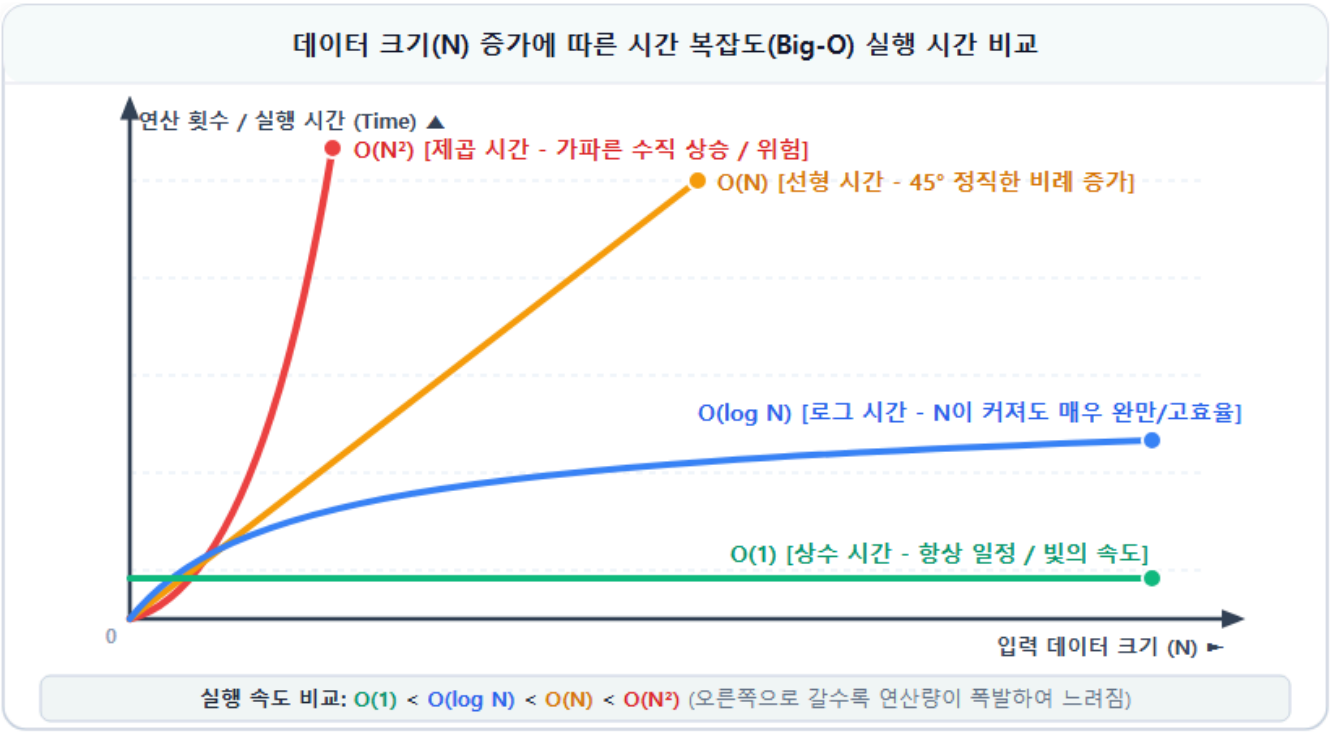
- **대표 알고리즘**: **버블 정렬 (Bubble Sort)**, **선택 정렬 (Selection Sort)**, **삽입 정렬 (Insertion Sort)**.

📊 한눈에 비교하는 빅오(Big-O) 속도 비교표

> **속도 순서 (빠름 ➔ 느림):**
> **O(1) < O(log N) < O(N) < O(N²)**

빅오 표기법	명칭	N=10일 때 연산 수	N=1,000일 때 연산 수	N=1,000,000 (백만)일 때 연산 수	실행 속도 체감 비유
O(1)	상수 시간	**1번**	**1번**	**1번**	⚡ 빛의 속도 (즉시 반환)
O(log N)	로그 시간	**약 3번**	**약 10번**	**약 20번**	🚀 로켓의 속도 (이진 탐색)
O(N)	선형 시간	**10번**	**1,000번**	**1,000,000번**	🚗 정직한 자동차 속도 (1중 루프)
O(N²)	제곱 시간	**100번**	**1,000,000번** (백만 번)	**1,000,000,000,000번 (1조 번!)**	🐢 렉 걸리는 거북이 (2중 루프)

📈 시간 복잡도 곡선 그래프 (Time Complexity Graph)



> 💡 **시험 1초 판별 꿀팁:**
>
> 1. 루프가 없고 단순 계산/배열 조회만 있다? ➔ **O(1)**
> 2. 루프가 돌 때마다 변수가 반(/2)으로 쪼개진다? ➔ **O(log N)**
> 3. 단순 루프가 1번(N번) 돈다? ➔ **O(N)**
> 4. 루프 안에 루프가 또 있는 2중 반복문이다? ➔ **O(N²)**

💡 2. 완전 탐색 (브루트 포스, Brute Force)

- **개념**: '무식한 힘'이라는 어원처럼, 문제를 해결하기 위해 **가능한 모든 경우의 수를 하나도 빠짐없이 100% 전부 확인**하는 탐색 기법입니다.
- **비유**: 4자리 숫자 자물쇠를 풀기 위해 `0000`부터 `9999`까지 1만 개의 조합을 전부 맞춰보는 것.
- **특징**: 무조건 정답을 찾을 수 있지만, 데이터의 양이 커지면 계산 시간이 기하급수적으로 늘어납니다.

💡 3. 백트래킹 (퇴각 검색, Backtracking)

- **개념**: 여러 갈래 길을 탐색해 나가다가, **원하는 조건이 아니거나 막다른 길에 도달하면 즉시 포기하고 이전 갈림길로 되돌아가서(Back)** 다른 경로를 시도하는 효율적인

탐색 기법(가지치기, Pruning)입니다.
- **대표 예시**: 미로 탈출 길찾기, 스도쿠 빈칸 채우기, 8-퀸(N-Queen) 문제.

✍️ 단원 마무리 실전 연습 문제

[문제 1]

크기가 무제한인 빈 스택(Stack)이 있습니다. 다음과 같은 연산을 순서대로 수행했을 때, 마지막에 스택에서 꺼내지는(Pop) 숫자는 무엇입니까?

1. `PUSH(10)`
2. `PUSH(20)`
3. `POP()`
4. `PUSH(30)`
5. `PUSH(40)`
6. `POP()`
7. `POP()`

① 10
② 20
③ 30
④ 40

[문제 2]

다음 슈도코드를 실행한 후 화면에 출력되는 `CNT`의 최종 값은 얼마입니까?

```
CNT = 0
X = 16
WHILE (X > 1) {
    X = X / 2
    CNT = CNT + 1
}
PRINT(CNT)
```

[문제 3]

다음과 같이 정의된 함수 `calc(N)` 이 있습니다.

```
FUNCTION calc(N) {
    SUM = 0
    FOR (I = 1 to N) {
        IF (I % 2 == 1) {   // I가 홀수이면 더함
            SUM = SUM + I
        }
    }
    RETURN SUM
}
```

`calc(6)` 을 호출했을 때의 최종 반환값(Return Value)은 얼마입니까?

① 6

② 9

③ 12

④ 15

🎯 정답 및 해설

[문제 1 정답] ③

- [해설]
  - 1. Push(10) ➔ 스택: `[10]` (10 저장)
  - 2. Push(20) ➔ 스택: `[10, 20]` (20 저장)
  - 3. Pop() ➔ 맨 위 20 꺼냄, 남은 스택: `[10]`
  - 4. Push(30) ➔ 스택: `[10, 30]`
  - 5. Push(40) ➔ 스택: `[10, 30, 40]`
  - 6. Pop() ➔ 맨 위 40 꺼냄, 남은 스택: `[10, 30]`
  - 7. Pop() ➔ 맨 위 **30** 꺼냄! (스택에 남은 것은 `[10]`)
  - 따라서 마지막 7번 연산에서 스택에서 꺼내지는 숫자는 **30**입니다.

[문제 2 정답] 4

- [해설]
  - X가 16에서 시작하여 2로 계속 나누어집니다:
    - 1회차: X = 16 / 2 = 8, CNT = 1 (8 > 1 참)
    - 2회차: X = 8 / 2 = 4, CNT = 2 (4 > 1 참)
    - 3회차: X = 4 / 2 = 2, CNT = 3 (2 > 1 참)
    - 4회차: X = 2 / 2 = 1, CNT = 4 (1 > 1 **거짓 ➔ 탈출**)
  - 출력값: **4** (16은 2^4이므로 4번 나누어야 1이 됩니다).

[문제 3 정답] ②

- [해설]
  - `calc(6)`을 호출하면 I가 1부터 6까지 순회하며 홀수일 때만 SUM에 누적합니다:
    - I = 1 (홀수): SUM = 0 + 1 = 1
    - I = 2 (짝수): 조건 불만족 (건너뜀)
    - I = 3 (홀수): SUM = 1 + 3 = 4
    - I = 4 (짝수): 조건 불만족 (건너뜀)
    - I = 5 (홀수): SUM = 4 + 5 = 9
    - I = 6 (짝수): 조건 불만족 (건너뜀)
  - 반복문 종료 후 최종적으로 `SUM` 값인 `9` **(②번)**가 반환(RETURN)됩니다.

🏆 디미고 소질·적성검사 100점 만점 달성 전략

이 문서는 기초 개념을 이미 모두 이해한 학생이, 실제 디미고 소질·적성검사에서 상위 1% 이내의 점수(만점)를 받기 위한 실전 전략 지침서입니다.

🎯 1. 100점을 가르는 '시간 압박' 극복 훈련

시험은 **30문항 / 50분**입니다. OMR 마킹 시간(약 5분)을 제외하면 실제 문제를 푸는 시간은 45분, 즉 **1문제당 1분 30초**입니다.

💡 구체적 훈련 방법

1. **양치기(반복 훈련)의 힘**: 논리 퍼즐이나 경우의 수 문제는 많이 풀어볼수록 '패턴'이 보입니다. 처음엔 5분 걸리던 문제가 비슷한 유형을 10번 풀면 1분 안에 풀립니다.
2. **조건 반사**: 진법 변환이나 스택/큐 추적 문제 같은 단순 계산/알고리즘 문제는 문제를 보자마자 기계적으로 손이 움직이도록 연습해야 합니다. 여기서 시간을 단축해야 어려운 퍼즐 문제에 3~4분을 투자할 수 있습니다.
3. **손으로 푸는 속도 높이기**: 눈으로 풀다 보면 중간에 변수 값을 까먹거나 조건이 꼬여서 처음부터 다시 읽어야 합니다. 이게 가장 큰 시간 낭비입니다. 무조건 표를 그리고 숫자를 적어내려가며 푸세요.

🎯 2. '버리기' 전략 (Skipping Strategy)

만점을 노리는 학생이 가장 흔히 하는 실수는 "모든 문제를 다 풀어야 한다"는 압박감에 한 문제에 5~10분을 낭비하는 것입니다.

💡 실전 행동 강령

1. **30초 룰**: 문제를 읽고 30초 안에 풀이 방법(접근법)이 떠오르지 않는다면, 일단 과감하게 별표(★)를 치고 다음 문제로 넘어가세요.
2. **'만점 방지용 킬러 문항' 피하기**: 디미고 시험에는 출제자가 의도적으로 시간을 잡아먹게 만든 극악의 퍼즐 문제가 1~2문제 반드시 섞여 있습니다. 이 문제들을 맨 마지

막에 푸는 것이 100점을 향한 가장 똑똑한 전략입니다.

3. **2회독 풀이법**:
    - 1회독 (1번~30번): 1분 안에 풀리는 쉬운 문제만 빠르게 다 풀기 (약 20분 소요)
    - 2회독: 별표 친 문제들 중 만만해 보이는 것부터 차근차근 풀기 (약 15분 소요)
    - 3회독: 남은 1~2개의 킬러 문항에 남은 시간 모두 쏟아붓기

🎯 3. 킬러 문항(융합형 문제) 대비

기초 교재에서는 논리 추론, 이산수학, 컴퓨팅 사고력을 따로 배웠지만, 고난도 문제는 이것들이 섞여서 나옵니다.

💡 융합 문제의 예시

- **[논리 + 수학]**: A, B, C가 달리기 시합을 했는데, 1등이 2진수로 1010 이라 하고, 2등이 8진수로 12 라고 거짓말을 할 때 진짜 순위는?
- **[수학 + 컴퓨팅]**: 5개의 섬이 다리로 연결된 그래프(이산수학)에서, 시작점에서 출발해 모든 섬을 방문하는 최단 경로를 찾는 알고리즘(컴퓨팅) 문제.

이런 문제들은 당황하지 않고, 배운 개념들을 **하나씩 차례대로 적용**하여 풀이를 쪼개는 능력(분할 정복)이 필요합니다.

👉 이제 마음의 준비가 되었다면, 다음 장인 **[비버챌린지 집중 공략]**으로 넘어가 당신의 한계를 테스트해 보세요!

🦫 제1특강: 비버 챌린지 집중 공략

> **컴퓨팅 사고력의 끝판왕, 비버 챌린지 유형 완벽 마스터!**

비버 챌린지(Bebras Challenge)는 전 세계적으로 열리는 정보(SW) 능력 평가입니다. 디미고 적성검사의 '컴퓨팅 사고력' 문제는 이 대회의 기출문제와 매우 유사한 패턴을 보입니다.

💡 핵심 공략 1: 그래프와 네트워크 최적화

주로 마을(노드)과 길(간선)이 주어지고, "가장 적은 비용으로 모든 마을을 연결하려면?" 혹은 "A마을에서 B마을로 가는 가장 빠른 길은?" 을 묻는 문제입니다.

👉 행동 강령:

1. **욕심쟁이(Greedy) 전략**: 길의 길이(비용)가 적혀 있다면, 무조건 가장 숫자가 작은 선부터 하나씩 연결해 봅니다. (단, 사이클이 생기게 연결하면 안 됩니다!)
2. 숫자를 직접 간선 옆에 적어가면서 덧셈 실수를 하지 않는 것이 중요합니다.

💡 핵심 공략 2: 상태 변환과 패턴 (오토마타)

"기계 A에 동전을 넣고 빨간 버튼을 누르면 사탕이 나오고 기계 상태가 B로 변한다" 같은 문제입니다.

👉 행동 강령:

1. 기계의 현재 '상태(State)'가 무엇인지 파악하세요.
2. 입력(버튼 누르기 등)이 들어올 때마다 화살표를 따라가며 현재 위치에 동그라미를 치세요.
3. 절대로 머리속으로만 암기하며 따라가지 마세요.

💡 핵심 공략 3: 암호화와 복호화

규칙을 주고 평문을 암호문으로 바꾸거나(암호화), 그 반대(복호화)를 요구합니다.

👉 **행동 강령:**

1. 시저 암호(알파벳 N칸 밀기)인지, 문자 위치 바꾸기인지 패턴을 파악하세요.
2. 알파벳 표(A~Z)를 시험지 귀퉁이에 빠르게 적어두고 손가락으로 짚어가며 푸는 것이 훨씬 빠르고 정확합니다. (A B C D E F G H I J K L M N O P Q R S T U V W X Y Z)

🎯 대표 기출 변형 훈련

[문제 1] 마법의 구슬 변환기
마법의 변환기에 빨간 구슬을 넣으면 파란 구슬 2개로 변합니다. 파란 구슬을 넣으면 노란 구슬 1개와 빨간 구슬 1개로 변합니다. 노란 구슬을 넣으면 구슬이 사라집니다.
처음에 빨간 구슬 1개를 넣고, 기계에서 더 이상 구슬이 변하지 않을 때까지 작동시켰을 때, 최종적으로 남는 구슬은 어떤 색깔, 몇 개일까요?

> **풀이:**
>
> 1. 빨간색(1) -> 파란색(2)
> 2. 파란색(2) -> 노란색(2) + 빨간색(2)
> 3. 노란색(2)은 사라짐. 남은 건 빨간색(2).
> 4. 빨간색(2) -> 파란색(4)
> 5. 파란색(4) -> 노란색(4) + 빨간색(4)
> 6. ... 끝이 나지 않고 계속해서 개수가 2배씩 늘어나는 '무한 루프(Infinite Loop)'에 빠집니다!
>
> **정답: 기계는 멈추지 않고 계속해서 구슬을 만들어낸다.**

[문제 2] 비버의 통나무 쌓기 (스택 응용)
비버가 강가에 A, B, C 세 개의 기둥을 세우고 통나무를 쌓고 있습니다.
크기가 서로 다른 3개의 통나무(큰, 중간, 작은)가 A 기둥에 '큰-중간-작은' 순서로 위로 쌓여 있습니다.
한 번에 맨 위에 있는 통나무 하나만 다른 기둥으로 옮길 수 있고, 큰 통나무는 작은 통나무 위에 올릴 수 없습니다. (하노이의 탑 규칙)
모든 통나무를 C 기둥으로 그대로 옮기려면 최소 몇 번 이동해야 할까요?

풀이:
1개 옮길 때: 1번
2개 옮길 때: 3번
3개 옮길 때: 7번
공식: $2^n - 1$
정답: 7번

(더 많은 문제는 공식 비버챌린지 사이트의 기출문제를 참고하세요!)

📊 제2특강: 정렬 알고리즘 (Sorting) 특강

> **뒤죽박죽 섞인 숫자들을 크기 순서대로 빠르게 줄 세우는 마법!**

정렬(Sorting)은 컴퓨터가 데이터를 찾기 쉽게 오름차순(작은 것부터)이나 내림차순(큰 것부터)으로 정리하는 과정입니다. 컴퓨팅 사고력 시험에서는 정렬 코드를 짜라고 하지 않습니다. 대신 **정렬이 진행되는 중간 과정을 손으로 그릴 수 있는가?**를 봅니다.

가장 대표적이고 시험에 잘 나오는 2가지 정렬 방법을 완벽히 이해해 봅시다.

---

1. 버블 정렬 (Bubble Sort)

> **옆 사람과 키를 비교해서, 내가 더 크면 자리를 바꾼다!**

버블 정렬은 거품이 물 위로 뽀글뽀글 올라오는 것처럼, **가장 큰 숫자가 맨 뒤로 밀려나는** 방식입니다.

💡 작동 원리 (오름차순 기준)

1. 첫 번째 숫자와 두 번째 숫자를 비교합니다. 앞 숫자가 더 크면 둘의 자리를 바꿉니다(Swap).
2. 두 번째 숫자와 세 번째 숫자를 비교합니다. 앞이 더 크면 바꿉니다.
3. 이 과정을 맨 끝까지 반복하면, **가장 큰 숫자가 맨 끝자리로** 가게 됩니다. (이것을 '1회전(Pass 1) 완료'라고 합니다.)
4. 끝자리는 이미 가장 큰 숫자로 고정되었으니 제외하고, 나머지 숫자들로 다시 처음부터 1~3 과정을 반복합니다.

📝 실전 예제 추적하기

배열: **[5, 3, 4, 1]** (이 숫자들을 버블 정렬로 오름차순 정렬해 봅시다.)

[1회전 (Pass 1)]

- 5와 3 비교 -> 5가 크니까 바꿈: [**3, 5**, 4, 1]

- 5와 4 비교 -> 5가 크니까 바꿈: [3, **4, 5**, 1]
- 5와 1 비교 -> 5가 크니까 바꿈: [3, 4, **1, 5**]
  👉 1회전 완료! 가장 큰 '5'가 맨 끝으로 고정됨. (현재 상태: 3, 4, 1, **5**)

[2회전 (Pass 2)] (맨 끝 5는 제외하고 3, 4, 1만 비교)

- 3과 4 비교 -> 4가 크니까 그대로: [**3, 4**, 1, 5]
- 4와 1 비교 -> 4가 크니까 바꿈: [3, **1, 4**, 5]
  👉 2회전 완료! 두 번째로 큰 '4'가 고정됨. (현재 상태: 3, 1, **4, 5**)

[3회전 (Pass 3)] (3과 1만 비교)

- 3과 1 비교 -> 3이 크니까 바꿈: [**1, 3**, 4, 5]
  👉 정렬 완료! 최종 상태: **[1, 3, 4, 5]**

> ⚠️ **시험 단골 질문**: "버블 정렬 2회전이 끝난 직후의 배열 상태는?" -> 정답: [3, 1, 4, 5]

---

2. 선택 정렬 (Selection Sort)

> **남은 숫자들 중에서 가장 작은 놈을 '선택'해서 맨 앞으로 데려온다!**

선택 정렬은 아주 직관적입니다. 훑어보고 제일 작은 걸 맨 앞자리와 바꾸는 방식입니다.

💡 작동 원리 (오름차순 기준)

1. 전체 숫자 중에서 **가장 작은 숫자**를 찾습니다.
2. 그 숫자를 **맨 첫 번째 자리**의 숫자와 자리를 바꿉니다(Swap).
3. 이제 맨 첫 자리는 가장 작은 숫자로 고정되었습니다.
4. 첫 번째 자리를 제외한 나머지 숫자들 중에서 다시 가장 작은 숫자를 찾아, 두 번째 자리와 바꿉니다. 이를 반복합니다.

📝 실전 예제 추적하기

배열: **[5, 3, 4, 1]**

[1회전 (Pass 1)]

- 전체(5, 3, 4, 1) 중에서 가장 작은 숫자는 '1'입니다.
- '1'을 맨 첫 번째 자리에 있는 '5'와 자리를 바꿉니다.
  👉 1회전 완료! (현재 상태: **1**, 3, 4, 5)

[2회전 (Pass 2)]

- 고정된 1을 제외하고 나머지(3, 4, 5) 중에서 가장 작은 숫자는 '3'입니다.
- '3'은 이미 두 번째 자리에 있으므로 자기 자신과 바꿉니다 (사실상 이동 없음).
  👉 2회전 완료! (현재 상태: **1, 3**, 4, 5)

[3회전 (Pass 3)]

- 나머지(4, 5) 중 가장 작은 숫자는 '4'입니다.
- 자리가 그대로 유지됩니다.
  👉 정렬 완료! 최종 상태: **[1, 3, 4, 5]**

🎯 최소 이동(Swap) 횟수 구하기 문제

알고리즘 없이 사람의 머리로 풀 때, 최소 몇 번 맞바꿔야 정렬이 끝나는지 묻는 퀴즈입니다.

> **문제**: [4, 1, 3, 2] 를 오름차순[1, 2, 3, 4]으로 만들려면 최소 몇 번 두 숫자의 자리를 맞바꿔야 할까요?
>
> **풀이 (눈으로 보고 자기 자리가 아닌 것끼리 직접 교환)**:
>
> 1. '4'는 맨 끝으로 가야 하고 '2'는 두 번째로 와야 하니, 4와 2를 바꿉니다.
>   -> [2, 1, 3, 4] (교환 1회)
> 2. '2'와 '1'의 자리만 바뀌면 완벽하네요. 둘을 바꿉니다.
>   -> [1, 2, 3, 4] (교환 2회)
>   **정답**: 2번

🚀 완벽 정복 팁

정렬 문제는 눈으로 풀면 반드시 헷갈립니다. **문제지 여백에 대괄호 [ ] 를 그리고 숫자가 이동하는 과정을 한 줄 한 줄 직접 써 내려가는 것**이 실수를 없애고 만점을 받는 유일한 비결입니다!

🔌 제3특강: 필수 탐색 알고리즘 및 논리 회로 특강

> **100점을 위해 반드시 넘어야 할 마지막 산! 탐색, 회로, 압축, 그리고 최적화**

이 파일은 시험에 단골로 출제되지만 우리가 앞서 다루지 않았던 4가지 핵심 심화 개념을 다룹니다. (※ 모든 주요 개념에는 영어 용어를 병기했습니다. 컴퓨터 과학에서는 영어 용어 자체가 힌트가 될 때가 많습니다!)

---

1. 탐색 알고리즘 (Search Algorithm)

수많은 데이터 중에서 내가 원하는 특정 값을 찾아내는 방법입니다. 정렬(Sorting)과 함께 가장 중요한 주제입니다.

💡 선형 탐색 (Linear Search)

- **개념**: 처음부터 끝까지 순서대로 하나씩 확인하며 찾는 무식하지만 확실한 방법입니다.
- **비유**: 100장의 카드 더미에서 맨 위부터 한 장씩 뒤집으며 원하는 카드를 찾는 것.
- **특징**: 운이 좋으면 1번 만에 찾지만, 최악의 경우(Worst Case) 맨 마지막에 있어서 100번을 다 뒤집어야 할 수도 있습니다.

💡 이진 탐색 (Binary Search)

- **개념**: 데이터가 **반드시 크기 순서대로 정렬(Sorted)**되어 있을 때만 쓸 수 있습니다. 가운데 값을 찔러보고, 절반씩 버려가며 찾는 아주 빠른 방법입니다.
- **비유**: 술래잡기에서 "업(Up) / 다운(Down)" 게임을 하는 것과 똑같습니다. 1~100 사이의 숫자를 맞출 때 무조건 '50'부터 부르는 이유가 바로 이진 탐색입니다!
- **특징**: 100개의 데이터가 있어도 한 번 물어볼 때마다 절반(50 -> 25 -> 12 -> 6 -> 3 -> 1)이 날아가므로, 최악의 경우라도 단 **7번** 만에 무조건 답을 찾습니다.

---

2. 논리 회로 (Logic Gate)

컴퓨터의 뇌(CPU) 안에서 0과 1을 가지고 계산을 수행하는 문(Gate)입니다. 디미고 시험에서는 논리식을 이 기호로 그려놓고 최종 출력값을 묻습니다.

💡 꼭 외워야 할 3+1가지 기호 (그림을 상상하며 외우세요!)

1. **AND 게이트 (논리곱)**
    - **모양**: 알파벳 'D' 처럼 생겼습니다. (반듯하고 둥금)
    - **규칙**: 입력이 모두 1(참)일 때만 출력도 1이 나옵니다. (1 * 1 = 1)
2. **OR 게이트 (논리합)**
    - **모양**: 알파벳 'C'나 화살촉처럼 앞뒤가 모두 뾰족하게 휘어 있습니다.
    - **규칙**: 입력 중 하나라도 1(참)이면 출력은 1이 나옵니다. (1 + 0 = 1)
3. **NOT 게이트 (논리부정)**
    - **모양**: 삼각형 끝에 작은 동그라미(O)가 붙어 있습니다.
    - **규칙**: 입력을 반대로 뒤집습니다. (1을 넣으면 0, 0을 넣으면 1)
4. **XOR 게이트 (배타적 논리합, Exclusive OR)**
    - **모양**: OR 게이트 모양인데, 입력 쪽에 줄이 하나 더 그어져 있습니다.
    - **규칙**: 두 입력이 **서로 다를 때만** 1이 나옵니다. (입력이 1, 0 이면 출력은 1 / 1, 1 이면 출력은 0)

---

3. 데이터 압축 (Data Compression)

컴퓨터가 용량을 아끼기 위해 문자나 그림을 짧게 줄여서 표현하는 방법입니다.

💡 런-랭스 인코딩 (Run-Length Encoding, RLE)

- **개념**: 연속해서 반복되는 문자(Run)를 '문자 + 반복 횟수(Length)'로 묶어서 표현하는 압축 기법(Compression Technique)입니다.
- **예제**: `AAAAABBBCCDAA` 라는 13글자가 있습니다.
    - A가 5번 -> `5A` (또는 A5)
    - B가 3번 -> `3B`
    - C가 2번 -> `2C`
    - D가 1번 -> `1D`

    - A가 2번 -> `2A`
- **결과**: `5A3B2C1D2A` (10글자로 줄어듦!) 비버 챌린지(Bebras Challenge)에서 흰색 픽셀과 검은색 픽셀로 이루어진 그림을 압축할 때 이 규칙이 100% 출제됩니다.

---

4. 탐욕 알고리즘 (Greedy Algorithm)

미래를 생각하지 않고, **지금 이 순간 당장 가장 좋아 보이는 것**을 선택하는 알고리즘입니다. 최적화(Optimization) 문제에 자주 쓰입니다.

💡 거스름돈 문제 (Coin Change Problem)

- **상황**: 편의점에서 860원을 거슬러 줘야 하는데, 동전 개수를 **가장 적게** 주고 싶습니다. (500원, 100원, 50원, 10원이 무한정 있다고 가정)
- **그리디(탐욕) 선택**: 지금 당장 집을 수 있는 가장 큰(가장 탐나는) 동전부터 집습니다!
    1. 500원짜리 1개 선택 (남은 돈: 360원)
    2. 100원짜리 3개 선택 (남은 돈: 60원)
    3. 50원짜리 1개 선택 (남은 돈: 10원)
    4. 10원짜리 1개 선택 (남은 돈: 0원)
- **결과**: 총 6개의 동전. 이렇게 단위가 배수 관계일 때 탐욕 알고리즘은 항상 최고의 정답을 냅니다.

> 🎉 **축하합니다!** 이제 디미고 소질·적성검사에 출제되는 모든 유형의 알고리즘과 수학적 원리를 마스터했습니다. 남은 것은 끝없는 문제 풀이(양치기)뿐입니다!

📚 심화 학습을 위한 추천 외부 교재 및 사이트

제가 만들어드린 교재는 '이론과 문제 유형에 대한 직관'을 키워주는 가이드입니다.
시험 한 달 전부터는 엄청난 분량의 문제를 풀며 속도를 높이는 **양치기 훈련**이 필수입니다. 디미고 만점을 노리는 학생들을 위해 다음과 같은 시중 교재와 무료 사이트를 추천합니다.

---

1. 컴퓨팅 사고력 & 이산수학 훈련 사이트

🌟 비버 챌린지 코리아 (Bebras Challenge)

- **링크**: https://www.bebras.kr/
- **추천 이유**: 디미고 적성검사의 '컴퓨팅 사고력' 문제와 가장 흡사한 퀄리티와 유형을 자랑합니다.
- **활용법**: 웹사이트 내 '기출문제 체험하기' 메뉴에서 중학생 그룹(Group V, VI) 문제 위주로 타이머를 맞추고 풀어보세요.

🌟 한국정보올림피아드 (KOI) 1차 예선 기출

- **링크**: https://koi.or.kr/ (기출문제 게시판)
- **추천 이유**: 이산수학과 컴퓨팅 사고력을 가장 딥(Deep)하게 파고들 수 있습니다.
- **활용법**: 프로그래밍 문제(코딩)는 무시하시고, **'이산수학/컴퓨팅 사고력(객관식)' 부분만** 프린트해서 풀어보세요. 중등부 예선 1교시 문제들이 완벽한 대비가 됩니다.

---

2. 논리적 추론 및 상황 판단 교재 (시중 서적)

📘 PSAT (공직적격성평가) 상황판단영역 입문서

- **설명**: 5급 공무원이나 민간경력자를 뽑을 때 보는 시험이지만, 이 중 '상황판단' 영역의 논리퀴즈나 퍼즐 문제는 디미고 적성검사 심화 문제와 완벽히 겹칩니다.
- **추천 교재**:

    - 『민간경력자 PSAT 상황판단 기출문제집』(민간경력자 시험이 일반 PSAT보다 쉬워서 중학생이 입문하기 좋습니다.)
- **활용법**: 법률 지문을 묻는 문제는 건너뛰고, 오직 **'조건 추리', '참/거짓 퍼즐', '수리 계산 퍼즐'**만 골라서 풉니다.

📗 멘사(Mensa) 논리 퍼즐 시리즈

- **추천 교재**: 『멘사 논리 퍼즐』, 『멘사 추론 퍼즐』 등
- **추천 이유**: 딱딱한 시험 공부가 지루할 때, 머리를 식히면서도 완벽하게 시험 대비를 할 수 있습니다.

---

3. 학습 스케줄 조언 (D-30)

- **월/수/금**: 비버 챌린지 기출문제 1세트 풀고 오답 분석 (컴퓨팅 사고력 데이)
- **화/목**: 정보올림피아드 1차 객관식 기출 + PSAT 상황판단 논리퀴즈 풀이 (이산수학/논리 데이)
- **토/일**: 전체 복습 및 시간 제한(50분 30문항) 실전 모의고사 1회분 자체 실시

> **교재보다 중요한 것은, 낯선 문제를 만났을 때 당황하지 않고 그림을 그려가며 조건을 분해하는 태도입니다.**